FULL STACK DEVELOPMENT · FFSD PROJECT

PlayCarys Learning Guide

Understand every line of the platform you built — languages, buttons, wiring and flows.

Project Online Multiplayer Game Management Platform (OMGMP)

Stack Vanilla JS + HTML + CSS front-end · NestJS + TypeScript back-end · JSON flat-file database

Location D:\AKHIL\ACADEMIC\ALL sub\FFSD\codes\playcarys_updated

Scale 36 HTML pages · ~30,000 front-end lines · ~7,100 back-end lines · 12 core tables · 60+ API routes

Author Akhil

BEFORE YOU START

How to use this guide

You built this with AI assistance. This guide reverse-engineers it back into your head — the way you'd learn it if you had written it by hand.

The guide is built for **practical learning**. Every chapter follows the same shape:

1. **What it is** — one paragraph, plain English.
2. **Where it lives** — the exact file and line so you can open it side-by-side.
3. **The actual code** — the real lines from your project, not pseudo-code.
4. **Why it is written that way** — the design decision or the bug it prevents.
5. **Try it yourself** — a small experiment that proves you understood it.

The one rule for using this document Keep VS Code open next to this PDF. Every `file.js:120` reference is a real place you can jump to. Reading code you can't see is memorisation; reading code you *can* see is understanding.

Colour code used throughout

Marker	Meaning
FE	Front-end — runs in the browser (HTML / CSS / vanilla JavaScript)
BE	Back-end — runs on Node.js (NestJS / TypeScript)
DB	Data — localStorage, the in-memory store, or `live_database.json`
RT	Real-time — Socket.IO events or the polling fallback
SEC	Security — a rule that exists because something was exploitable without it

Why blue boxes A blue box explains a *design decision*: why the code chose option A over the obvious option B.

Why red boxes A red box records a *real bug* that existed in this project and the fix that removed it. These are the most valuable paragraphs in the guide — they are where the architecture came from.

Why green boxes A green box is something to *do*: run it, break it, watch it fail, fix it.

Suggested reading order

If you have…	Read
1 hour	Parts 1, 2, 4 and 5. That gives you the stack, the map, the universal button pattern and login.
1 day	Parts 1–8. You will be able to explain any screen in the app and trace any click to the database.
1 week	Everything, plus the exercises in Part 15. That is viva-proof understanding.

CONTENTS

Table of contents

Part 1 — Running the project
- 1.1 The two servers
- 1.2 The launcher script, line by line
- 1.3 Playing across two laptops

Part 2 — Languages and the stack
- 2.1 Every language, and exactly where it appears
- 2.2 Every library and what it does for you
- 2.3 Why vanilla JS and not React

Part 3 — The folder map
- 3.1 Front-end tour
- 3.2 Back-end tour
- 3.3 The 36 pages and who can open them

Part 4 — The data layer
- 4.1 The 12-table schema
- 4.2 The three copies of your data
- 4.3 DB.save() — the heart of the app
- 4.4 Merge, append-only tables and tombstones
- 4.5 Back-compat getters

Part 5 — How every button works
- 5.1 The universal 6-step recipe
- 5.2 A button dissected end to end
- 5.3 The render-from-state pattern
- 5.4 Why `onclick=` and not addEventListener

Part 6 — Authentication
- 6.1 The five roles
- 6.2 Login traced click by click
- 6.3 Registration (player and publisher)
- 6.4 Guest mode
- 6.5 Signed tokens — HMAC by hand
- 6.6 One account, one device
- 6.7 Presence heartbeat
- 6.8 Logout and password change

Part 7 — Authorisation
- 7.1 RolesGuard, decorator, metadata
- 7.2 Why the role header was a hole
- 7.3 Front-end route protection

Part 8 — The complete button catalogue
- 8.1 Landing page
- 8.2 Player pages (12)
- 8.3 Publisher pages (6)
- 8.4 Moderator pages (4)
- 8.5 Admin pages (6)
- 8.6 Superuser pages (7)
- 8.7 Global buttons on every page

Part 9 — End-to-end flows
- 9.1 Sign in
- 9.2 Publish a game
- 9.3 Admin review and approve
- 9.4 Create room → play → record result
- 9.5 Quick Play matchmaking
- 9.6 Tournament: create → bracket → advance
- 9.7 Report → moderator action → ban
- 9.8 Notifications and friends

Part 10 — NestJS back-end anatomy
- 10.1 The 5-file module pattern
- 10.2 Bootstrap, pipes, guards, interceptors
- 10.3 DTOs and validation
- 10.4 Persistence and atomic writes
- 10.5 Swagger

Part 11 — Real-time layer
- 11.1 The gateway and its 12 events
- 11.2 PlayCarysLive and the synthetic storage event
- 11.3 Polling fallback

Part 12 — Game delivery pipeline
- 12.1 Three delivery kinds
- 12.2 Bundle upload and zip safety
- 12.3 The static security scanner
- 12.4 The iframe wrapper and the SDK

Part 13 — Domain rules reference
- 13.1 All 50+ rules by module

Part 14 — Security decisions
- 14.1 Seven holes and their fixes
- 14.2 What is still open

Part 15 — Practice
- 15.1 25 exercises in difficulty order
- 15.2 Viva questions with answers
- 15.3 File index

PART 1

Running the project

Before you read a single line of code, get it running. Everything in this guide is easier to believe when the app is on screen next to it.

1.1 The two servers

PlayCarys is **two independent programs** that talk over HTTP. Neither one can do the job alone.

Server	Port	Started by	Serves
FE **Static web server**	`8080`	`npm start` in `front-end/` — runs `http-server`	All the `.html`, `.css`, `.js` and game folders. It is a dumb file server: no logic at all.
BE **NestJS API**	`3000`	`npm run start:dev` in `back-end/`	60+ REST routes under `/api/v1`, the Socket.IO gateway, Swagger docs at `/api/docs`, and writing `live_database.json`.

```
Browser  ──HTTP GET──▶  http-server :8080   ──▶  returns dashboard.html, utils.js, db.js …
   │
   ├──fetch /api/v1/db-export──▶  NestJS :3000  ──reads──▶  in-memory arrays (store.ts)
   ├──fetch /api/v1/db-import──▶  NestJS :3000  ──writes──▶  live_database.json  (atomic)
   └──websocket ws://:3000    ──▶  GameGateway  ──broadcast──▶  every other browser
```

The full request picture. Port 8080 never touches data; port 3000 never serves HTML.

Starting it manually

```
# Terminal 1 — the API
cd back-end
npm install          # first time only
npm run start:dev    # nest start --watch, recompiles on save

# Terminal 2 — the website
cd front-end
npm install          # first time only
npm start            # http-server . -p 8080 -c-1
```

Then open `http://localhost:8080`.

> **Why `-c-1` in the http-server command** `-c-1` disables caching. Without it the browser holds on to an old `utils.js` and your edits appear to do nothing — the single most common "my change didn't work" during development. See `front-end/package.json:6`.

1.2 The launcher script, line by line

`front-end/run.bat` automates all of the above and does five extra things. Read it once — it is genuinely good engineering and explains several architectural choices.

What it does	Why it matters
Derives all paths from `%~dp0`	The folder can be renamed, moved or copied to another laptop and the script still works. Nothing is hardcoded.
Checks ports 3000 and 8080 with `Get-NetTCPConnection`	If a previous run is still listening it offers to kill it. This is why you rarely see "EADDRINUSE".
Runs `npm install` **if** `node_modules` **is missing**	Fresh clone works with a double-click.
Starts `npx localtunnel --port 8080`	Creates a public URL so a friend anywhere in the world can play. Written to `public_url.txt`.
Finds this laptop's Wi-Fi IPv4 address	Prints `http://192.168.x.x:8080` so a second laptop on the same Wi-Fi can join.

1.3 Playing across two laptops — the auto-detect trick

This is one of the smartest lines in the whole project. Open `front-end/data/db.js:34`:

```
window.PC_BACKEND_PORT = 3000;
window.PC_BACKEND = (function () {
  // 1. Explicit override in the URL:  ?server=10.0.99.135
  const qs = new URLSearchParams(window.location.search).get('server');
  if (qs) { /* …persist to localStorage and use it… */ }

  // 2. Previously saved override
  const saved = localStorage.getItem('pc_backend_url');
  if (saved) return saved;

  // 3. Autodetect: same host as the page, backend port.
  const host = window.location.hostname || 'localhost';
  return 'http://' + host + ':' + window.PC_BACKEND_PORT;
})();
window.PC_API = window.PC_BACKEND + '/api/v1';
```

Why derive the API address instead of hardcoding it If `API_BASE` were the literal string `http://localhost:3000`, then when your friend opens `http://192.168.1.7:8080` on *their* laptop, their browser would try to reach *their own* localhost:3000 — where nothing is running. By reading `window.location.hostname`, whichever machine served the page is also treated as the API host. **Any laptop can be the server with zero code edits.**

Try it Start the app. Open DevTools console and type `window.PC_API`. Now open the same page as `http://127.0.0.1:8080` and check again — the value follows the address bar. Then try `http://localhost:8080/?server=1.2.3.4` and watch it save the override to localStorage (and break the app, because 1.2.3.4 has no backend — clear it with `localStorage.removeItem('pc_backend_url')`).

The three ways to reach the app

Address	Who	Requirement
`http://localhost:8080`	You, host laptop	Nothing.
`http://192.168.x.x:8080`	Friend on the same Wi-Fi	Same network; Windows Firewall must allow Node on ports 3000 and 8080.
localtunnel URL	Anyone on the internet	Host laptop stays on and online. URL changes every launch.

Gotcha worth remembering The friend's browser fetches **both** the page (8080) and the API (3000) from your laptop. If only 8080 is allowed through the firewall, the site loads but nothing saves and everything looks read-only. That symptom — "page opens but login says cannot reach the server" — is always a port 3000 problem.

PART 2

Languages and the stack

Six languages appear in this project. Here is each one, where it lives, how much of it there is, and what job it does.

2.1 Every language, and exactly where it appears

Language	Volume	Where	What it does here
HTML5	36 files ~14,000 lines	`index.html` `pages/<role>/*.html`	Page structure. Every page is **self-contained**: markup, its own `<style>` block, and its own `<script>` block, plus three shared files it pulls in.
CSS3	2 files ~3,965 lines	`css/variables.css` (3,150) `css/landing.css` (815) + per-page `<style>`	The whole design system: CSS custom properties for colours/spacing, then component classes (`.btn` , `.card` , `.badge` , `.modal-overlay` , `.drawer-panel`) plus keyframe animations.
JavaScript (ES6+)	~13,000 lines	`js/utils.js` (2,615) `data/db.js` (2,571) `js/game-launch.js` (262) + inline page scripts	All front-end behaviour. Plain browser JavaScript — no framework, no build step, no bundler. Uses `async/await` , template literals, `Map` / `Set` , spread, optional chaining, getters.
TypeScript	69 files 7,141 lines	`back-end/src/**`	The entire API. Interfaces define the schema (`store.ts`), decorators define routes, DTO classes define validation.

JSON	—	`live_database.json`
		`package.json` , `manifest.json` ,
		`games/.bundles.json`
			The database itself, dependency manifests, the PWA manifest and the game-bundle sidecar index.

Batch (.bat)	1 file, ~150 lines	`front-end/run.bat`	The launcher. Also embeds small **PowerShell** one-liners for port and IP discovery.

The shape of a typical page

```
<!DOCTYPE html>
<head>
  <link rel="stylesheet" href="../../css/variables.css">   ← design system
  <script src="../../data/db.js"></script>                 ← the data + DB.save()
  <script src="../../js/utils.js"></script>                ← Auth, Toast, Modal, navbar…
  <script src="../../js/game-launch.js"></script>          ← only on game pages
  <style> /* page-specific CSS */ </style>
</head>
<body>
  <div id="navbar"></div>              ← filled by buildNavbar()
  <div class="page-wrapper"> …markup… </div>
  <script> /* page logic: render(), handlers */ </script>
</body>
```

Load order is not optional `db.js` must load before `utils.js` , because `utils.js` immediately references `DB` and `window.PC_API` (which `db.js` defines at line 54). Both must load before the page script, which calls `Auth.requireAuth()` on the first line of its boot function. Swap any two and you get `ReferenceError: DB is not defined` .

2.2 Every library and what it does for you

BE Back-end dependencies — `back-end/package.json`

Package	Version	Job in this project
`@nestjs/core` `@nestjs/common`	^11.1	The framework. Provides `@Module`, `@Controller`, `@Injectable`, dependency injection, the request pipeline (pipes → guards → interceptors → handler).
`@nestjs/platform-express`	^11.1	Makes Express the HTTP engine underneath Nest. Also gives you `FileInterceptor` (multer) used for the game `.zip` upload.
`@nestjs/websockets` `@nestjs/platform-socket.io` `socket.io`	^11.1 / ^4.8	The real-time layer. `@WebSocketGateway` in `game.gateway.ts` handles 12 event types.
`class-validator` `class-transformer`	^0.14 / ^0.5	Powers the DTOs. `@IsString()`, `@IsInt()`, `@Min(0)` on a DTO class + the global `ValidationPipe` = automatic 400 responses for bad input.
`@nestjs/swagger` `swagger-ui-express`	^11.3 / ^5.0	Auto-generates interactive API docs from your decorators at `/api/docs`, and dumps `docs/swagger.json` on boot.
`rxjs`	^7.8	Nest's internal observable stream. You touch it directly only in `persistence.interceptor.ts` (`.pipe(tap(...))`).
`reflect-metadata`	^0.1	The engine behind decorators. `Reflect.defineMetadata` is how `@Roles('admin')` attaches data that `RolesGuard` later reads.

FE Front-end dependencies — `front-end/package.json`

Package	Job
`http-server`	The only one that actually runs during normal use. Serves the folder on port 8080.
`express`, `cors`, `body-parser`	Used only by the legacy `front-end/server.js` — a 50-line flat-file DB server that predates the NestJS back-end. **Not used any more** (it would also clash on port 3000). Keep it as a historical artefact or delete it.

Two things load from the internet **1.** Google Fonts (Orbitron + Inter) via `@import` at the top of `css/variables.css`. **2.** Socket.IO client from `cdn.socket.io`, injected at runtime by `initPlayCarysWebSockets()` (`js/utils.js:90`). Offline, fonts fall back to system sans and the socket never connects — which is exactly why `PlayCarysLive.startPolling(5000)` exists as a backup. The app degrades; it does not break.

2.3 Why vanilla JS and not React

Understanding this choice explains almost every pattern you will meet later.

Concern	What React would give you	What this project does instead
Rendering	Virtual DOM diffing, components re-render on state change	`element.innerHTML = array.map(...).join('')` — full re-render of a section, by hand.
State	`useState`, Context, Redux	One global `DB` object + `localStorage` + the server. Any file can read or write it.
Events	`onClick={handler}` compiled to real listeners	Inline `onclick="fnName('id')"` in HTML strings, resolved against `window` at click time.
Routing	React Router, single page	Real page navigation — `window.location.href = 'x.html'` — through the `navigate()` wrapper.
Build	Vite/Webpack, JSX transpile, HMR	**None.** Save the file, hit refresh. No compile step exists on the front-end.

The trade you made **You gained:** zero build complexity, no `node_modules` in the browser, every file readable top to bottom, and full visibility of exactly what the browser executes. For a course project that has to be *explained* in a viva, this is a genuine advantage.

You paid: manual re-renders (forget one and the screen goes stale — that is the origin of `PlayCarysLive` and the `playcarys:dbupdate` event), no compile-time type safety on the front-end, and string-built HTML which needs care about quoting and escaping.

PART 3

The folder map

Where everything lives, and — more usefully — what to open first when you are hunting something down.

3.1 Front-end tour

```
front-end/
├── index.html          1,749 lines — landing page + ALL login/registration logic
├── manifest.json       PWA install metadata (name, icons, theme colour)
├── sw.js               Service worker: cache-on-install, network-first, cache fallback
├── run.bat             The launcher
├── live_database.json  ★ THE DATABASE. One JSON file. All 21 tables.
├── server.js           Legacy 50-line flat-file server — superseded, not used
│
├── css/
│   ├── variables.css   3,150 lines — design tokens + every component class
│   └── landing.css     815 lines — landing page only
│
├── data/
│   ├── db.js           ★ 2,571 lines — seed data, DB.save(), merge, tournament engine
│   └── auth.js         2 lines — dead file, ignore
│
├── js/
│   ├── utils.js        ★ 2,615 lines — Auth, Presence, Toast, Modal, Validate, navbar, drawer
│   ├── game-launch.js  262 lines — "can this game be played, and where?"
│   ├── playcarys-sdk.js82 lines — the SDK third-party games embed
│   ├── api.js          44 lines — thin fetch wrapper (lightly used)
│   └── stores/authStore.js 117 lines — normalise/denormalise user shapes
│
├── pages/
│   ├── player/    12 pages      ├── admin/     6 pages
│   ├── publisher/ 6 pages       └── superuser/ 7 pages
│   └── moderator/ 4 pages
│
├── games/              Playable builds — 2 builtin + 4 uploaded bundles
│   ├── chess/          builtin, game_id 7, 3,058-line single file
│   ├── hill_climbing/  builtin, game_id 8
│   ├── hexgl-16/  2048-18/  hextris-19/  neon-braker-20/   ← unpacked .zip uploads
│   └── .bundles.json   sidecar: which game_id owns which folder
│
└── images/             Game thumbnails and PWA icons
```

The three files that matter most If you only ever read three files, read `data/db.js`, `js/utils.js` and `back-end/src/app.controller.ts`. Together they are the entire nervous system of the app.

3.2 Back-end tour

```
back-end/
├── src/
│   ├── main.ts              109 — bootstrap: pipes, CORS, guard, interceptor, Swagger
│   ├── app.module.ts        34  — registers all 12 feature modules
│   ├── app.controller.ts    ★ 367 — db-export / db-import. The sync brain.
│   │
│   ├── data/
│   │   ├── store.ts         ★ 439 — all interfaces + the in-memory arrays + seed data
│   │   ├── persistence.ts   163 — debounced atomic writes to live_database.json
│   │   └── persistence.interceptor.ts 34 — schedules a save after every successful write
│   │
│   ├── auth/       login, register, change-password, HMAC token util
│   ├── roles/      RolesGuard + @Roles decorator
│   ├── users/      CRUD, status changes, role assignment
│   ├── game/       ★ largest module: listing, bundle upload, zip, scanner, review
│   ├── game-session/ rooms, quick-play, join/leave/end + the Socket.IO gateway
│   ├── tournament/ create, register, brackets
│   ├── match-result/ read-only match history
│   ├── report/     player reports
│   ├── moderator-action/ warnings, restrictions, bans
│   ├── notification/ direct messages and role broadcasts
│   ├── player-profile/ points, leaderboard, match history
│   └── publisher-profile/ studio stats
│
├── dist/       compiled JS — never edit, regenerated by nest build
├── docs/       swagger.json, auto-written on every boot
└── .auth-secret ★ auto-generated 96-char HMAC key. Do not commit. Do not delete.
```

Never edit `dist/` `dist/` is the TypeScript compiler's output. `npm run start:dev` overwrites it on every save of a `src/` file. Edits there vanish and confuse you for an hour. Always edit `src/` .

3.3 The 36 pages and who can open them

Role	Landing page after login	All pages
Player	`player/dashboard.html`	dashboard · games · tournaments · my-tournaments · profile · leaderboard · match-history · match-details · join-session · room-lobby · play · notifications
Publisher	`publisher/dashboard.html`	dashboard · publish · publish-tournament · mygames · manage-genres · profile
Moderator	`moderator/dashboard.html`	dashboard · reports · sessions · bans
Admin	`admin/dashboard.html`	dashboard · hub · users · approvals · game-review · settings
Superuser	`superuser/hub.html`	hub · dashboard · users · admins · games · reports · settings

The mapping lives in `Auth.gotoDashboard()` at `js/utils.js:678` :

```
gotoDashboard() {
  const session = this.getSession();
  const roleMap = { superuser:'superuser', admin:'admin', moderator:'moderator',
                    publisher:'publisher', player:'player', guest:'player' };
  const roleDir = roleMap[session.role] || 'player';
  // superuser is the one role whose home is hub.html, not dashboard.html
  const targetFile = session.role === 'superuser' ? 'hub.html' : 'dashboard.html';
  window.location.href = getPathPrefix() + 'pages/' + roleDir + '/' + targetFile;
}
```

`getPathPrefix()` — the "how many `../` do I need" helper Pages sit at `/pages/player/x.html` but assets sit at `/css/` and `/games/` . `getPathPrefix()` (`js/utils.js:162`) counts the path segments after `/pages/` and returns the right number of `../` . From the landing page it returns `''` ; from a role page, `'../../'` . Every cross-folder link in the app goes through it.

PART 4

The data layer

This is the part of the project that most needs explaining, because it does something unusual: the same data exists in three places at once, and code exists specifically to keep them from destroying each other.

4.1 The 12-table schema

The project was designed from an SQL schema, then implemented without SQL. The schema still governs every field name. The comment header at `data/db.js:1` lists it in order.

#	Table	Primary key	Holds
1	`roles`	`role_id`	The five roles. Fixed, seeded, never grows.
2	`users`	`user_id`	Every account. `role_id` is the FK that decides everything else.
3	`player_profiles`	`user_id` (also FK)	IS-A extension of a user with `role_id 4`: carys_points, play_time, matches_won.
4	`publisher_profiles`	`user_id` (also FK)	IS-A extension for `role_id 5`: total/approved/active games, revenue_pcp.
5	`games`	`game_id`	Listing + delivery config + review state.
6	`tournaments`	`tournament_id`	Competition metadata, dates, prize, organiser.
7	`sessions`	`session_id`	A game room: host, max players, status, mode.
8	`match_results`	`(session_id, player_id)`	Composite key. One row per player per match. **Append-only.**
9	`reports`	`report_id`	Player-submitted complaints.
10	`moderator_actions`	`action_id`	Audit trail of every warning / restrict / ban / review.
11	`player_tournament`	`(player_id, tournament_id)`	Junction table — who registered for what.
12	`player_session`	`(player_id, session_id)`	Junction table — who is in which room.

Nine more tables were added later as features grew. They are **not** in the original schema but are equally real:

Table	Purpose
`bans`	Active and expired bans with level, duration, who issued it.
`notifications`	DMs, friend requests, room invites, role broadcasts. **Append-only.**
`tournament_matches`	The generated bracket — one row per match slot.
`tournament_results`	Final standings.
`deleted_tournaments` / `deleted_games`	**Tombstones.** "This absence is deliberate." Explained in §4.4.
`favorites`, `reviews`	The ♥ button and player star ratings.
`game_reviews`	Admin approval decisions — who approved which exact build. **Append-only.**
`settings`, `genres`, `activityLog`, `telemetry`, `live_events`, `quickPlayQueue`	Front-end-owned. The back-end has no model for these — see the passthrough mechanism in §10.4.

The IS-A relationship, in practice

```
// A player is a users row PLUS a player_profiles row, joined on user_id.
users:           { user_id: 7, user_name: "DragonSlayer", role_id: 4, email: …, status: "active" }
player_profiles: { user_id: 7, player_name: "DragonSlayer", carys_points: 4680, matches_won: 234 }

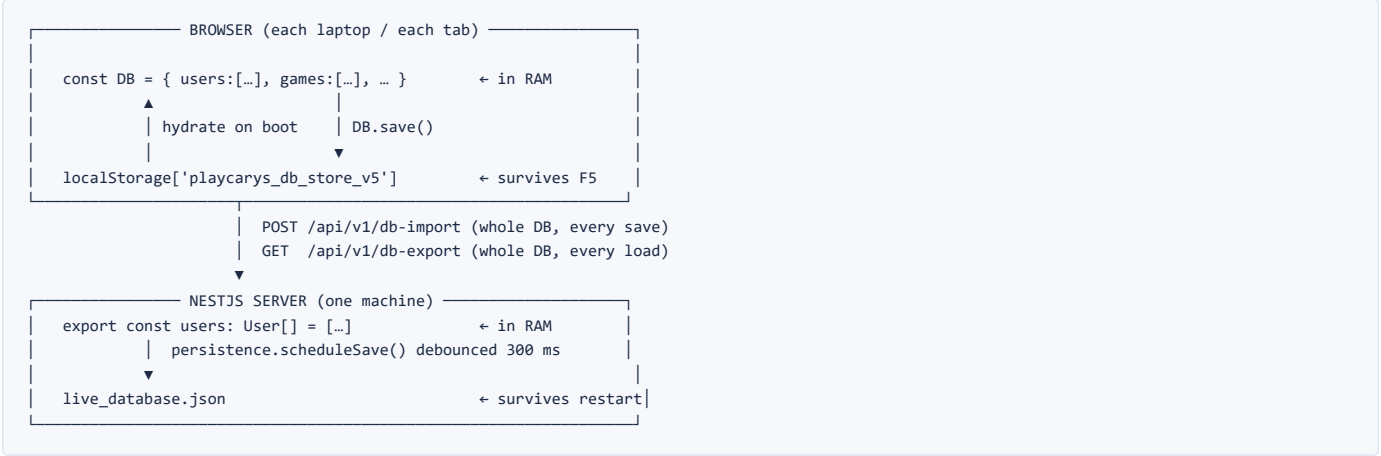
// So to show a player's points you always need BOTH lookups:
const u = DB.users.find(x => sameId(x.user_id, uid));
const p = DB.player_profiles.find(x => String(x.user_id) === String(uid));
```

Why the profile is auto-created at registration Rule **R-AUTH-5**. If a user row existed without its profile row, every leaderboard, points display and match-history page would silently show nothing for that account — and no error, because `.find()` returns `undefined`, not a crash. See

`auth.service.ts:165` .

4.2 The three copies of your data

This is the single most important diagram in the guide.



Copy	Lives in	Survives	Written by
FE `DB` object	Browser RAM	Nothing — gone on refresh	Direct assignment: `DB.games.push(x)`
DB localStorage	Browser disk, ~5 MB	Refresh, tab close, reboot	`DB.save()` and `DB.saveLocal()`
BE store.ts arrays	Node RAM	Nothing — gone on restart	`db-import` and the API handlers
DB `live_database.json`	Server disk	Everything	`persistence.ts` , debounced, atomic

Boot sequence — what happens when you open any page **1.** `db.js` runs and `DB` is the hardcoded seed data.
2. localStorage is read; if a saved store exists it replaces the seed (hydration).
3. `GET /db-export` fetches the server snapshot.
4. `DB.mergeRemoteState(remote)` folds the server copy in *without destroying local-only rows*.
5. `attachBackCompatGetters()` re-adds the derived properties.
6. The page's own `render()` runs and paints the screen.

4.3 `DB.save()` — the heart of the app

Every single write in the entire front-end ends with this call. `data/db.js:839` :

```
DB.save = function () {
  attachBackCompatGetters();

  // 1. Serialise every non-function property of DB
  const toSave = {};
  Object.keys(DB).forEach(key => {
    if (typeof DB[key] !== 'function') toSave[key] = DB[key];
  });
  const payload = JSON.stringify(toSave);

  // 2. Instant local persistence — never fails, never waits
  localStorage.setItem(DB_KEY, payload);
  _lastPayload = payload;

  // 3. Push the WHOLE database to the server
  _pendingSync = fetch(window.PC_API + '/db-import', {
    method: 'POST',
    headers: Object.assign({ 'Content-Type': 'application/json' },
                           Auth.apiAuthHeaders()),   // ← Bearer token if signed in
    body: payload
  }).then(res => {
    _pendingSync = null;
    if (!res.ok) { console.error('Backend rejected DB sync:', res.status); return; }

    // 4. Tell every OTHER browser that something changed
    if (window.playcarysSocket) window.playcarysSocket.emit('global_db_update');
  }).catch(err => { _pendingSync = null; /* offline — localStorage still has it */ });

  return _pendingSync;
};
```

Why it POSTs the entire database, not just the change This is unusual and worth being honest about in a viva. A production app would send `PATCH /games/17 {"rating": 4.5}`. This app sends all 21 tables every time.

The upside: the front-end needs zero knowledge of REST endpoints. Any page can mutate any array and call `DB.save()`. That is why 36 pages were built quickly.
The downside: two laptops saving at once can overwrite each other — *which is exactly the problem that the append-only union in §4.4 was written to solve.* Understand this trade and you understand half the codebase.

The three save variants — and when each is correct

Call	What it does	Use when
`DB.save()`	localStorage + POST to server + socket broadcast	Almost always. Any real change other players must see.
`DB.saveLocal()`	localStorage only, no network	On `beforeunload`, or when a network call would block navigation.
`DB.flushSync()`	`navigator.sendBeacon()` with the last payload	You are about to set `window.location.href`. A normal `fetch` is **cancelled** when the page navigates; `sendBeacon` is not.
`DB.whenSynced(ms)`	Promise that resolves when the POST lands, or after a timeout	Inside `navigateTo()` — wait for "the match started" to reach the server before leaving the page.

Real bug: the write that never arrived `Auth.logout()` releases the account's session claim, then immediately sets `window.location.href`. The browser cancels the in-flight `fetch` in the same tick, so the release never reached the server and the account stayed "claimed" — you could not sign in again until the 100-second stale timer expired.
Fix (`js/utils.js:617`): `try { DB.save(); DB.flushSync(); }` — belt and braces.

4.4 Merge, append-only tables and tombstones

Because every client uploads its whole database, a client holding a *stale* snapshot would delete newer rows. Three mechanisms prevent that.

Mechanism 1 — append-only tables (union, not replace)

Declared on both sides. Front-end at `data/db.js:681`, back-end at `app.controller.ts:104`:

```
const APPEND_ONLY = {
  match_results:   r => String(r.session_id) + '|' + String(r.player_id),
  sessions:        r => String(r.session_id || r.id),
  users:           u => (u.user_id != null && String(u.user_id) !== 'undefined')
                        ? 'id:' + String(u.user_id) : 'em:' + String(u.email),
  player_profiles: p => String(p.user_id),
  notifications:   n => String(n.id),
  telemetry:       r => String(r.id),
  highlights:      h => String(h.userId)+'|'+String(h.gameId)+'|'+String(h.time)
};
```

Each entry is a function that produces a **row identity**. During a merge, rows the sender did not include are **kept** rather than dropped:

```
// back-end, app.controller.ts:314 — the union
const seen      = new Set(body[table].map(keyOf));
const preserved = arr.filter(row => !seen.has(keyOf(row)));
arr.length = 0;
arr.push(...body[table], ...preserved);   // client rows + rows only the server had
```

Real bugs this fixed · A saved Hill Climbing run disappeared seconds later because an unrelated save from another tab replaced `match_results`.
· Four registered accounts were destroyed by any client whose `users` array was still the 12-row seed.
· Friend requests and chat messages vanished before the recipient ever saw them, because a presence heartbeat from the other laptop replaced `notifications`.

Why `users` is keyed on `user_id` and not email Email was tried first and caused a worse bug: email is **editable** from the profile page. Keying identity on a mutable field meant editing your email *forked your account into two rows* — the old row survived the union (it looked like a row the client had simply omitted) and still answered to the old username and password. **Lesson: never key an identity on a field the user can change.**

Mechanism 2 — tombstones

Append-only protection creates a new problem: if absence never means deletion, *how do you delete anything?* Answer — say so explicitly. `data/db.js:917`:

```
DB.deleteGame = function(gameId) {
  const sid = String(gameId);
  const idx = DB.games.findIndex(g => String(g.game_id ?? g.id) === sid);
  if (idx !== -1) DB.games.splice(idx, 1);                    // remove locally
  DB.deleted_games.push({ game_id: sid, deleted_at: new Date().toISOString() });  // ← the tombstone
};
```

The server then applies the tombstone list *after* the union (`app.controller.ts:330`), dropping any game that appears in it. `deleted_games` is itself append-only — otherwise a client that had never seen the deletion would send an empty list, wipe the tombstone, and a third client would resurrect the game.

Mechanism 3 — field-level rules inside a merged row

Even when two copies of the *same* row are merged, some fields must not simply take the server's value. `data/db.js:795`:

```
Object.keys(remoteRow).forEach(field => {
  // Never write through a getter with no setter — the update would vanish silently.
  const d = Object.getOwnPropertyDescriptor(localRow, field);
  if (d && d.get && !d.set) return;

  // lastSeen only ever moves FORWARD (otherwise presence flaps Online↔Offline)
  if (field === 'lastSeen' && newerStamp(localRow.lastSeen, remoteRow.lastSeen) === localRow.lastSeen) return;

  // readBy / clearedBy are per-user state on a SHARED row → union, never overwrite
  if ((field === 'readBy' || field === 'clearedBy') && Array.isArray(remoteRow[field])) {
    const union = new Set((localRow[field] || []).map(String));
    remoteRow[field].forEach(x => union.add(String(x)));
    localRow[field] = [...union];
    return;
  }
  localRow[field] = remoteRow[field];
});
```

And why fields are copied instead of swapping the object Pages hold live references into these arrays — `room-lobby.html` keeps `currentSession` pointing at a `DB.sessions` row for the whole lobby. Since a merge now runs every few seconds, replacing the row object would leave the page mutating a **detached copy**: the host sets `status='in-progress'` on an orphan, and `DB.save()` writes the untouched original. Copying field-by-field keeps every existing reference valid.

4.5 Back-compat getters

The project has two naming conventions living side by side: SQL-style `game_id`, `game_name`, `game_approval_status`, and UI-style `id`, `title`, `status`. Rather than rename 36 pages, `attachBackCompatGetters()` defines **derived, non-enumerable properties**:

```
Object.defineProperties(r, {
  id:           { get() { return String(r.report_id); }, enumerable: false, configurable: true },
  reporterName: { get() {
                    const u = DB.users.find(u => String(u.user_id) === String(r.reported_by));
                    return u ? (u.user_name || u.username) : 'Unknown';
                  }, enumerable: false, configurable: true },
  desc:         { get() { return r.reason || ''; }, enumerable: false, configurable: true }
});
```

Property	Effect
`get()`	Computed fresh on every read — `reporterName` follows a username change automatically.
`enumerable: false`	**Critical.** `JSON.stringify()` skips non-enumerable properties, so getters are never written to the database. Without this, derived values would be persisted and could go stale.
`configurable: true`	Lets `attachBackCompatGetters()` be re-run safely after every merge.

The trap this creates `report.id = 99` does **nothing** — a getter with no setter silently swallows the assignment in non-strict mode. That is precisely why the merge code checks `if (d && d.get && !d.set) return;`. If you ever find "my assignment isn't sticking", check whether the property is a getter.

Try it In the console on any page: `DB.reports[0].reporterName` → a username. Now `Object.keys(DB.reports[0])` → `reporterName` is **not** in the list. Then `JSON.stringify(DB.reports[0])` → no `reporterName`. That is `enumerable:false` doing its job.

PART 5

How every button works

There is exactly one pattern. Learn it once here and all 400+ buttons in the project become readable at a glance.

5.1 The universal 6-step recipe

```
① HTML has an inline handler          <button onclick="doThing('7')">Do it</button>
            |
            ▼
② A global function runs              function doThing(id) { …
            |
            ▼
③ Validate / guard                      if (!x) { Toast.error('…'); return; }
            |
            ▼
④ Mutate the DB object                  DB.games.find(…).game_status = 'enabled';
            |
            ▼
⑤ Persist                               DB.save();      ← localStorage + server + socket
            |
            ▼
⑥ Re-render + feedback                  render(); Toast.success('Done!');
```

That is it. Every button — from "Ban player" to "Mark all read" to "Start Tournament" — is these six steps. Differences are only in *which* array is touched and *what* gets re-rendered.

5.2 A button dissected end to end

Take the publisher's **Enable / Disable** toggle on `pages/superuser/games.html`.

Step ① — the markup (generated, not static)

```
grid.innerHTML = list.map(g => `
  <div class="card">
    <div class="game-card-title">${g.title}</div>
    ${badgeHTML(g.status)}
    <button class="btn btn-sm" onclick="toggleGame('${g.id}')">
      ${g.game_status === 'enabled' ? '⏸ Disable' : '▶ Enable'}
    </button>
  </div>`).join('');
```

Read this carefully — it is the core front-end idiom The HTML is a **template literal** (backticks) with `${}` interpolation, produced by `.map()` over an array and glued with `.join('')`. The whole string is assigned to `.innerHTML`, which makes the browser parse it into real DOM. The `onclick` attribute is just text at that moment; it only becomes executable code when the element is clicked, and it is resolved against `window`.

Step ②–⑥ — the handler

```
function toggleGame(id) {
  // ② find the row — note String() on both sides: ids may be number OR string
  const g = DB.games.find(x => String(x.game_id ?? x.id) === String(id));
  if (!g) { Toast.error('Game not found'); return; }              // ③ guard

  // ③ domain rule R-GAME-4: only an APPROVED game may be enabled
  if (g.game_approval_status !== 'approved') {
    Toast.error('Only approved games can be enabled.'); return;
  }

  // ④ mutate
  g.game_status = (g.game_status === 'enabled') ? 'disabled' : 'enabled';

  // ⑤ persist — localStorage, then POST, then broadcast
  DB.save();

  // ⑥ repaint + tell the human
  render();
  Toast.success(`Game ${g.game_status}`);
}
```

What happens after `DB.save()`, invisibly

```
this browser          NestJS :3000               other browsers
    |                     |                           |
    ├─ localStorage ✓     |                           |
    ├─ POST /db-import ──▶ union-merge into store.ts    |
    |                     ├─ interceptor → scheduleSave()|
    |                     |      └─ 300 ms later ──▶ live_database.json (atomic)
    ├─ socket.emit('global_db_update') ──▶ gateway broadcast ──▶ 'global_db_update_received'
    |                     |                           ├─ PlayCarysLive.pull()
    |                     |                           ├─ GET /db-export
    |                     |                           ├─ DB.mergeRemoteState()
    |                     |                           └─ dispatch 'playcarys:dbupdate' → page re-renders
```

Prove it to yourself Open the app in two browser windows side by side, signed in as different accounts. Click Enable on one. Watch the other update within a second or two *without a refresh*. Now stop the back-end (`Ctrl+C` in terminal 1) and click again — the first window still works (localStorage) but the second never learns. That is the whole architecture in one experiment.

5.3 The render-from-state pattern

Almost every page follows the same skeleton. Learn it and you can navigate any of the 36.

```
let currentFilter = 'all';                    // ← page-local UI state

function render() {
  // 1. READ from the global DB
  let list = DB.games.filter(g => String(g.game_publisher) === String(myId));

  // 2. APPLY UI state (filters, search, sort, pagination)
  if (currentFilter !== 'all') list = list.filter(g => g.game_approval_status === currentFilter);

  // 3. HANDLE the empty case explicitly
  if (!list.length) { grid.innerHTML = emptyState('No games yet', 'Publish one', "navigate('publish')"); return; }

  // 4. BUILD one big HTML string and assign it once
  grid.innerHTML = list.map((g, i) => `…card markup…`).join('');
}

function setFilter(v) { currentFilter = v; render(); }   // ← filter buttons just change state + re-render

document.addEventListener('DOMContentLoaded', () => {
  const session = Auth.requireAuth(['publisher']);   // ← gate first, always
  if (!session) return;
  buildNavbar('publisher', 'mygames');
  render();
});
document.addEventListener('playcarys:dbupdate', render);  // ← live updates from other clients
```

Why a full `innerHTML` replace instead of updating one element It is O(everything) and it destroys DOM state (an open dropdown, a focused input, scroll position inside the list). But it is **impossible to get out of sync**: the screen is always a pure function of the data. Where that cost mattered the code does use surgical updates instead — see `_smoothRemoveNotif()` at `js/utils.js:2354`, which animates one notification out rather than repainting the drawer, and `_drawerFriendsRender()`, which refuses to re-render while someone is typing in a chat box.

5.4 Why `onclick=` in a string and not `addEventListener`

This will be asked. The honest answer:

Approach	Consequence here
`el.addEventListener('click', fn)`	Every `innerHTML =` destroys the old elements **and their listeners**. You would have to re-attach after every render, for every card, every time.
Event delegation on a parent	The correct professional answer. One listener, `e.target.closest('[data-action]')`, survives re-renders.
Inline `onclick` (what this project uses)	The handler travels *with* the HTML string, so re-rendering can never break the wiring. Simplest thing that works with the `innerHTML` pattern.

The cost you must be aware of The handler is a **string** that gets parsed as JavaScript. So a username containing an apostrophe breaks it — which is why you will see this defensive escaping in `js/utils.js:1269`: `showUserProfile('${String(session.username).replace(/'/g, "\\'")}', event)`. It also means every handler must be a **global** function (`window.foo = …` or a top-level `function foo`) — a function defined inside another function is invisible to an inline handler.

Exercise Find three places in the project where `event.stopPropagation()` appears inside an `onclick` (hint: `player/games.html`, the ♥ favourite button). Work out what would happen without it — the click would bubble to the card's own `onclick` and open the game detail as well as toggling the heart.

PART 6

Authentication

The most-asked-about subsystem, and the one that changed the most. Six mechanisms work together: password check, signed token, browser session, single-device claim, device handoff and presence heartbeat.

6.1 The five roles

role_id	Name	Can self-register?	Power
1	`superuser`	No	Everything. `RolesGuard` short-circuits: a valid superuser token passes *every* route.
2	`admin`	No	Approve games, manage users, platform settings, danger-zone actions.
3	`moderator`	No	Reports, bans, live session monitoring.
4	`player`	**Yes**	Play, join rooms, tournaments, report, friends.
5	`publisher`	**Yes**	Publish games and tournaments, view analytics.
—	`guest`	Auto	Not a DB role at all — a synthetic front-end-only session for browsing. Never persisted.

Rule R-AUTH-4 — and why it is enforced twice Registration allows only role 4 or 5 (`auth.service.ts:130`). But `db-import` can also *create* users, so the same clamp is repeated at `app.controller.ts:172` . Without the second copy, an attacker would not bother promoting an existing account — they would simply invent a brand-new one with `role_id: 1` . **A rule enforced at one entry point is not enforced.**

6.2 Login traced click by click

User types a username and password into the Player column of the login panel and clicks Sign In.

① The click → `doPanelLogin` — `index.html:1428`

```
<button class="btn btn-primary" onclick="doPanelLogin('${inputId}','${passId}','${loginRole}')">
```

Three arguments: the id of the username input, the id of the password input, and the role this panel is for (`'player'` , `'publisher'` , `'moderator'` , `'admin'` or `'superuser'`).

② Find the account locally

```
const user = DB.users.find(u => {
  const name = u.username || u.user_name;
  const uNameMatch  = name && name.toLowerCase() === inputVal.toLowerCase();
  const dbPhone     = u.phone || u.phone_number;
  const uPhoneMatch = dbPhone && dbPhone.replace(/[\s-]/g,'').toLowerCase() === cleanInput;
  // publishers may also sign in with their phone number
  return uNameMatch || (expectedRole === 'publisher' && uPhoneMatch);
});
if (!user) { showPanelError('Account not found.'); return; }
```

③ Local pre-checks (fast rejections, no network)

```
if (user.status === 'banned')    { showPanelError('Your account has been banned.'); return; }
if (user.status === 'suspended' && user.role !== 'player') { … return; }

// Strict per-panel role check: you cannot sign into the Admin column with a player account
if (userRole !== expectedRole) { showPanelError(`This login is for ${expectedRole} accounts only.`); return; }
```

④ The password is checked by the server — `index.html:1394`

```
async function verifyPasswordWithServer(user, password) {
  const body = { password };
  if (user.email) body.email = user.email; else body.username = user.user_name;
  if (user.role_id != null) body.role_id = Number(user.role_id);   // disambiguate shared emails

  const res = await fetch(window.PC_API + '/auth/login', {
    method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(body)
  });
  if (res.ok) {
    const data = await res.json();
    if (data.token) Auth.setApiToken(data.token);   // ← keep the signed token
    return 'ok';
  }
  // distinguish banned from wrong-password from server-unreachable
  …
  return 'bad';   // or 'banned' or 'offline'
}
```

Real vulnerability, now fixed The original code compared `user.password !== pEl.value` **in the browser**. For that to work, `GET /db-export` had to ship every account's *plaintext password* to every visitor — including the superuser's — on every page load, on an endpoint that requires no authentication.
Fix: `app.controller.ts:42` now strips it: `users.map(({password, ...safe}) => safe)`, and the comparison moved server-side to `POST /auth/login`.
Second vulnerability, same line The check used to read `if (user.password && user.password !== pEl.value)`. Note the `&&`: an account row carrying **no** password skipped the comparison entirely, so *any* password signed you in as that user — including staff accounts created outside the sign-up form. A missing password is now always a failure.

⑤ Server side — `AuthService.login()`, `auth.service.ts:29`

```
login(dto: LoginDto) {
  // An email may hold BOTH a player and a publisher account (R-AUTH-2),
  // so resolving by email can return several candidates.
  let candidates = this.authRepository.findAllByEmail(dto.email);
  if (dto.role_id != null) { const scoped = candidates.filter(u => u.role_id === dto.role_id);
                             if (scoped.length) candidates = scoped; }

  // The PASSWORD is what selects between them — checked BEFORE the ban test on purpose.
  const user = candidates.find(u => u.password === dto.password);
  if (!user) throw new UnauthorizedException('Invalid email or password');

  if (user.status === 'banned') throw new UnauthorizedException('Account is banned.');  // R-AUTH-3

  this.authRepository.updateLastSeen(user.user_id);
  const roleName = roles.find(r => r.role_id === user.role_id)?.role_name ?? 'player';

  return { user_id: user.user_id, username: user.user_name, email: user.email,
           role: roleName, role_id: user.role_id, avatar: user.avatar, status: user.status,
           token: signToken(user, roleName) };   // ★ the server signs the role
}
```

Why the ban check comes AFTER the password check Subtle and deliberate. With several accounts sharing an email, "which account is this?" can only be answered once the password matches. Checking status first could report *another* account's ban to someone signing into their own.

⑥ Single-session claim and device handoff

```
const outcome = await Auth.signInWithHandoff(user);   // may take up to 15 s — button is disabled meanwhile
if (!outcome.ok) { showPanelError(messages[outcome.reason]); return; }
```

Detailed in §6.6.

⑦ Store the browser session, push the claim, navigate

```
Auth.login(user, outcome.token);     // writes sessionStorage + localStorage
DB.save();                           // ★ push the claim to the server — see warning below
Toast.success(`Welcome back, ${user.username}!`);
setTimeout(async () => {
  await DB.whenSynced(1500);         // don't leave before the POST lands
  window.location.href = rolePages[user.role];
}, 800);
```

The "signed out immediately after signing in" bug Without that `DB.save()`, the freshly-minted session token existed only in this tab's memory. The dashboard then hydrated from the backend, saw the *old* token on the account row, and `checkSessionOwnership()` logged you straight back out

with "signed in on another device" — while you were the only person logged in anywhere. The comment in the source literally says **"Do not remove."**

6.3 Registration

Two forms: player (`doRegister` , `index.html:1563`) and publisher (`doPublisherRegister` , line 1628). Same six steps.

```
// 1. Declarative validation via the shared Validate helper
const valid = Validate.form([
  { el: uEl, rules: [{type:'required'},{type:'minLen',n:3},{type:'maxLen',n:20}] },
  { el: eEl, rules: [{type:'required'},{type:'email'}] },
  { el: pEl, rules: [{type:'required'},{type:'password'}] },     // 8+ chars, upper+lower+digit
  { el: cEl, rules: [{type:'required'},{type:'match', target: pEl}] }
]);
if (!valid) return;

// 2. Uniqueness — username globally, email PER ROLE
if (DB.users.find(u => u.username?.toLowerCase() === username.toLowerCase())) { …taken… }
if (DB.emailTakenForRole(emailVal, 'player')) { …taken… }

// 3. Create user + its IS-A profile row together (R-AUTH-5)
const numId = Date.now();                    // ← id strategy: timestamp
DB.users.push({ user_id:numId, user_name:username, role_id:4, id:String(numId),
                username, email:emailVal, password:pEl.value, role:'player',
                avatar: username.slice(0,2).toUpperCase(), status:'active', … });
DB.player_profiles.push({ user_id:numId, player_name:username, carys_points:100,
                          play_time:0, matches_won:0, player_status:true });

// 4. Claim, save, log in, redirect
const claim = Auth.claimSession(newUser);
DB.save();
Auth.login(newUser, claim.token);
```

Rule R-AUTH-2 — email is unique *per role*, not globally One person may reasonably have a player account **and** run a studio, using the same inbox. Nothing signs in with email (login matches on username, or a publisher's phone), so a shared address creates no ambiguity anywhere. **Username stays globally unique**, because that *is* the login identifier. Implemented in `DB.emailTakenForRole()` , `data/db.js:954` .

Something to notice Registration writes straight into `DB.users` and syncs via `db-import` — it does **not** call `POST /auth/register` , even though that endpoint exists and enforces the same rules. Both paths work because the same clamps are duplicated in `db-import` . If you refactored, routing registration through the API would be the cleaner design.

6.4 Guest mode

```
function guestLogin() {
  const guestUser = {
    id: generateId('u'),                                  // e.g. "u_l3k9x2" — a STRING id
    username: 'Guest_' + Math.floor(Math.random()*10000),
    role: 'guest', avatar: 'G', status: 'active', carysPoints: 0
  };
  Auth.login(guestUser);        // ← note: no DB.users.push, no DB.save
  window.location.href = 'pages/player/dashboard.html';
}
```

A guest exists **only** in `sessionStorage` . It is never a row in `users` . On logout, `Auth.logout()` cleans up after it (`js/utils.js:597`):

```
if (session.role === 'guest') {
  DB.sessions      = DB.sessions.filter(s => String(s.hostId) !== uid && String(s.created_by) !== uid);
  DB.player_session = DB.player_session.filter(ps => String(ps.player_id) !== uid);
  DB.save();
}
```

Why guests get string ids Registered users have numeric `user_id` s; guests get `"u_abc123"` . Mixing the two broke every `===` comparison in the codebase — a guest id `parseInt()` s to `NaN` , and `NaN === NaN` is `false` . That is why the helper `sameId(a,b)` (`js/utils.js:1309`) exists and is used everywhere: `String(a) === String(b)` . If you write a new comparison, **use** `sameId` .

6.5 Signed tokens — HMAC by hand

`back-end/src/auth/token.util.ts` is 143 lines and worth reading in full. It implements a JWT-like token with **no external library**, using Node's built-in `crypto` .

```
format:  base64url(payload) "." base64url(HMAC-SHA256(payload, SECRET))

payload: { sub: user_id, role_id, role: 'admin', iat: <ms>, exp: <ms + 12h> }
```

```
export function signToken(user, roleName) {
  const now = Date.now();
  const payload = { sub: user.user_id, role_id: user.role_id, role: roleName,
                    iat: now, exp: now + TOKEN_TTL_MS };        // 12 hours
  const body = b64url(JSON.stringify(payload));
  return `${body}.${signature(body)}`;
}

function signature(body) {
  return b64url(crypto.createHmac('sha256', SECRET).update(body).digest());
}
```

Verification — three separate defences

```
export function verifyToken(token) {
  if (!token) return null;
  const body = token.slice(0, dot), provided = token.slice(dot+1);
  const expected = signature(body);

  // ① CONSTANT-TIME comparison. A normal === leaks how many bytes matched via timing.
  //    Lengths are checked first because timingSafeEqual THROWS on a length mismatch,
  //    and that throw would itself be a timing signal.
  const a = Buffer.from(provided), b = Buffer.from(expected);
  if (a.length !== b.length || !crypto.timingSafeEqual(a, b)) return null;

  const payload = JSON.parse(fromB64url(body).toString('utf8'));
  if (typeof payload?.exp !== 'number' || Date.now() > payload.exp) return null;   // ② expiry
  if (typeof payload.sub !== 'number' || typeof payload.role !== 'string') return null; // ③ shape
  return payload;
}
```

Three deliberate design decisions in this file — all quotable in a viva **1. "A JWT in spirit but deliberately not one."** A real JWT has three segments and an `alg` header field. Libraries that trust that field have caused famous vulnerabilities (`alg: none`). Here there is **one** algorithm and the token cannot negotiate it.

2. The secret is never hardcoded. `AUTH_SECRET` env var wins; otherwise 48 random bytes are generated once and written to `back-end/.auth-secret` with mode `0600` . A shipped constant secret is the same as no signature at all.

3. The key is persisted to a file, not regenerated per boot. A fresh key each restart would invalidate every session — which to a user looks exactly like "the app randomly logs me out".

Practical consequence Delete `back-end/.auth-secret` and every signed-in staff user must log in again. Never commit it to git.

Where the token is stored and sent

```
// FRONT-END — js/utils.js:351
Auth.setApiToken(token)   →  sessionStorage + localStorage under 'playcarys_api_token'
Auth.apiAuthHeaders()     →  { Authorization: 'Bearer <token>' }  (or {} when signed out)

// attached to every DB.save() POST and every staff API call:
headers: Object.assign({ 'Content-Type': 'application/json' }, Auth.apiAuthHeaders())
```

6.6 One account, one device

Three fields on the `users` row implement this: `active_session_token` , `active_session_at` , plus the constant `SESSION_STALE_MS = 100000` (100 s).

```
Someone signs in.  Is the account already claimed?
│
├─ NO (no token, or last heartbeat > 100 s old)
│     └─ show "Sign in here?" confirmation  →  forceClaimSession()  →  done
│
└─ YES, and heartbeat is fresh
      └─ socket: 'device_login_request' → the OLD device's live tab
            ├─ old device renders a banner and immediately ACKs
            │     ├─ human clicks Confirm →  'approved' → new device claims, old device logs out
            │     ├─ human clicks Cancel  →  'denied'   → new device refused (15 s cooldown)
            │     └─ no answer in 15 s    →  'timeout'  → refused (a human DID see it and ignored it)
            └─ no ACK within 2.5 s        →  'unreachable' → fall back to the confirmation dialog
```

Why 'unreachable' is treated differently from 'timeout' `timeout` means a device **got** the prompt and chose not to answer — a real "no". `unreachable` means the server has **no live socket** for this account: dead battery, closed laptop. Treating that as refusal locked real owners out of their own accounts indefinitely — and the claim can still look *fresh*, because `Presence.beat()` keeps stamping `active_session_at` over HTTP from any tab whose socket has since dropped. This was hit in real testing.

The "randomly kicked out while just browsing" bug `Auth.login()` writes the session to **both** storages, but `sessionStorage` is per-tab. A second tab signing in minted token T2 while tab 1 still held T1 — and `Presence.beat()` has every tab re-assert its token every 45 s. The two tabs fought; whichever beat last owned the account, and the other logged itself out with "signed in on another device".

Fix (`js/utils.js:649`): `getSession()` now compares the two storages and, for the **same account**, adopts the newer token instead of competing with it. A *different* account in localStorage is left alone — that is a genuine account switch.

6.7 Presence heartbeat

`Presence` (`js/utils.js:696`) answers "is this person actually here right now?"

Constant	Value	Meaning
`ONLINE_MS`	2 min	Seen within this window → Online
`SKEW_MS`	3 min	How far into the future a timestamp may sit and still count as "now" (two laptops never agree on the clock)
`SESSION_LIVE_MS`	3 hours	How long an in-progress session may count as genuinely being played
heartbeat interval	45 s	Comfortably inside the 2-minute window without hammering the server

```
beat(push = true) {
  const u = DB.users.find(x => sameId(x.user_id ?? x.id, session.userId));
  u.lastSeen = new Date().toISOString();
  if (session.sessionToken) {                    // re-assert ownership of the account
    u.active_session_token = session.sessionToken;
    u.active_session_at    = new Date().toISOString();
  }
  push ? DB.save() : DB.saveLocal();
}
```

Three bugs solved by three small guards in `statusOf()` · **Everyone read "Online" forever** — the friend list stored a frozen snapshot of `user.status`, which is the *account* state (active/banned), not presence. Now resolved live at render time.

· **Future-dated accounts were pinned Online** — several rows carry 2027 dates, making `age` negative, and negative is `<= ONLINE_MS` . Now: within `SKEW_MS` → treat as now; beyond → Offline.

· **Anyone who ever pressed Play read "Playing" forever** — solo runs are created `in-progress` and nothing closes them. Now `liveSessionOf()` also requires a start timestamp within 3 hours.

6.8 Logout and password change

```
logout(reason) {
  this.setApiToken(null);                    // ① drop the signed token FIRST
  // ② guest cleanup (rooms they hosted)
  // ③ release the account claim so you can sign in elsewhere immediately
  if (u.active_session_token === session.sessionToken) {
    u.active_session_token = null; u.active_session_at = null;
    DB.save(); DB.flushSync();               // ← sendBeacon, survives the navigation
  }
  // ④ carry the reason across the redirect so a Toast can explain it
  sessionStorage.setItem('pc_logout_reason', reason);
  sessionStorage.removeItem(this.SESSION_KEY);
  localStorage.removeItem(this.SESSION_KEY);
  window.location.href = getPathPrefix() + 'index.html';
}
```

Password change goes through the API, not `db-import` — `auth.service.ts:96` :

```
changePassword(dto) {
  const user = this.authRepository.findById(dto.user_id);
  // Identical response whether the account is missing OR the password is wrong,
  // so this cannot be used to enumerate which user ids exist.
  if (!user || user.password !== dto.current_password)
    throw new UnauthorizedException('Current password is incorrect');

  if (dto.new_password === dto.current_password) throw new BadRequestException('…must be different');

  user.password = dto.new_password;
  user.active_session_token = null;   // ★ evict every other device
  user.active_session_at    = null;
}
```

Rule R-AUTH-6 — why the current password is required Without it, anyone who reached an open session — a shared laptop, an unlocked machine, a borrowed phone — could lock the real owner out of their own account in two clicks. The profile page offered exactly that. Knowing the current password *is* the proof of identity, which keeps this endpoint honest even though the platform's legacy role header is client-asserted.

PART 7

Authorisation — who is allowed to do what

Authentication is "who are you". Authorisation is "may you". Three layers do it here, and only one of them is trustworthy.

7.1 RolesGuard, decorator, metadata

Three files, twelve lines of concept.

The decorator — `roles/roles.decorator.ts` (4 lines)

```
import { SetMetadata } from '@nestjs/common';
export const ROLES_KEY = 'roles';
export const Roles = (...roles: string[]) => SetMetadata(ROLES_KEY, roles);
```

`SetMetadata` attaches an array to the method using `reflect-metadata`. It **does nothing on its own** — it is a label that a guard later reads.

Applying it

```
@Patch(':id/approve')
@Roles('admin', 'superuser')          // ← the label
approveGame(@Param('id', ParseIntPipe) id: number, @Body() dto: ApproveGameDto) { … }
```

The guard — `roles/roles.guard.ts`

```
canActivate(context: ExecutionContext): boolean {
  const requiredRoles = this.reflector.getAllAndOverride<string[]>(ROLES_KEY,
                        [context.getHandler(), context.getClass()]);
  if (!requiredRoles) return true;               // ← no @Roles ⇒ route is PUBLIC

  const request = context.switchToHttp().getRequest();

  // ── 1. A SIGNED TOKEN IS AUTHORITATIVE ──────────────────
  const payload = verifyToken(bearerFrom(request.headers['authorization']));
  if (payload) {
    request.auth = payload;                      // handlers can trust this identity
    const role = String(payload.role).toLowerCase();
    if (role === 'superuser') return true;       // superuser passes everything
    if (requiredRoles.some(r => r.toLowerCase() === role)) return true;
    throw new ForbiddenException(`Requires one of: [${requiredRoles.join(', ')}]`);
  }

  // ── 2. NO TOKEN: the legacy header, for NON-PRIVILEGED roles only ──
  const incomingRole = String(request.headers['role'] ?? '').toLowerCase();
  if (PRIVILEGED.has(incomingRole))              // {superuser, admin, moderator}
    throw new ForbiddenException('staff access requires a signed session.');

  return requiredRoles.some(r => r.toLowerCase() === incomingRole);
}
```

7.2 Why the `role` header was a hole

The vulnerability, in one command Originally the guard trusted `request.headers['role']` — a value the caller writes themselves.

```
curl -X PATCH http://localhost:3000/api/v1/users/9 \
     -H 'role: admin' -H 'Content-Type: application/json' \
     -d '{"role_id":1}'
```

A player account promoted to **superuser** with one unauthenticated request. It was verified twice against this server — once through `PATCH /users/:id` with the forged header, once through the unauthenticated `db-import`.

The fix has three parts: (1) the role now comes from a payload the *server* signed at login; **(2)** the three privileged roles can *never* be granted by the header; **(3)** `db-import` pins `role_id` and `status` to what the server already holds unless the caller presents an admin/superuser token.

The `db-import` clamp, `app.controller.ts:205`:

```
if (!staffCaller && incoming.role_id !== existing.role_id) {
  console.warn(`[db-import] refused role change for user ${existing.user_id}`);
  incoming.role_id = existing.role_id;      // silently pinned back
  incoming.role    = existing.role;
}
if (!staffCaller && incoming.status !== existing.status) { incoming.status = existing.status; }
```

Why the legacy header is still accepted at all Player and publisher pages still authenticate client-side and were never migrated. Their header is accepted so the app keeps working — but it grants **nothing** that can reach another user's account or platform state, because `PRIVILEGED` is checked regardless of what the route asks for. This is a pragmatic, documented compromise, not an oversight. Being able to say that clearly is worth marks.

7.3 Front-end route protection

```
requireAuth(allowedRoles) {
  const session = this.getSession();
  if (!session) { window.location.href = getPathPrefix()+'index.html'; return null; }
  if (allowedRoles && !allowedRoles.includes(session.role)) {
    window.location.href = getPathPrefix()+'index.html'; return null;
  }
  return session;
}
```

Called as the first line of every page's boot function: `const session = Auth.requireAuth(['publisher']);`

This is convenience, not security It reads `sessionStorage`, which the user controls. Anyone can type `sessionStorage.setItem('playcarys_session', '{"role":"admin",…}')` and open the admin pages. **What stops them is that the pages are useless:** every privileged *action* goes to a guarded API route that demands a signed token, and the token cannot be forged without `.auth-secret`. **Front-end checks hide UI. Back-end guards enforce rules.** Never confuse the two.

Layer	Trustworthy?	Purpose
`Auth.requireAuth()`	**No**	Stops honest users seeing pages that would only confuse them.
`role` header	**No**	Legacy. Non-privileged roles only.
Signed Bearer token + `RolesGuard`	**Yes**	The actual security boundary.

PART 8

The complete button catalogue

Every clickable control in the project, page by page: what function it calls, which table it changes, and what the user sees afterwards. Use this as a lookup table while reading the code.

> **How to read these tables** **Button** = the visible label. **Calls** = the exact function name in the `onclick`. **Touches** = which `DB.*` array it mutates (`—` means it only changes the screen). **Then** = what happens after the mutation.
> Unless a row says otherwise, **every mutating button ends with `DB.save()` followed by a re-render and a Toast.**

8.1 Landing page — `index.html` (31 controls)

Button	Calls	Touches	Then
Player Login (nav)	`openLoginPanel('player')`	—	Opens the 5-column login overlay, pre-focused on the Player column.
Publisher Login	`openLoginPanel('publisher')`	—	Same overlay, Publisher column. Publishers may sign in with phone *or* username.
Team ▾ dropdown	`toggleTeamDropdown(event)` → `handleTeamItem('admin'|'moderator'|'about')`	—	Reveals the staff login columns (moderator / admin / superuser). Staff are not advertised on the public nav.
Admin / Superuser tab	`switchAdminTab('admin'|'superuser')`	—	Swaps the staff column between the two staff panels; re-renders via `colHTML(col)`.
Sign In (each column)	`doPanelLogin(uId, pId, role)`	`users`	The full flow of §6.2: local lookup → role check → `POST /auth/login` → device handoff → `Auth.login()` → `DB.save()` → redirect.
Fill demo credentials	`fillPanelDemo(...)`	—	Types a demo username into the inputs. Convenience for testing only.
Create Account	`openRegister()` → `doRegister()`	`users`, `player_profiles`	Validates 4 fields, checks username + per-role email uniqueness, pushes both rows, claims the session, redirects to the player dashboard.
Register Studio	`openPublisherRegister()` → `doPublisherRegister()`	`users`, `publisher_profiles`	Same, plus a country code + phone field validated by `Validate.phone`. Leading `0` is stripped.
Play as Guest	`guestLogin()`	— *(nothing persisted)*	Mints a synthetic `{role:'guest'}` session in storage only, redirects to the player dashboard.
Explore / About / Publishers	`document.getElementById(id).scrollIntoView({behavior:'smooth'})`	—	Pure anchor scrolling. No JS state at all.
Close ✕ / overlay click	`closeLoginPanel()`, `closePanelOnOverlay(e)`, `Modal.close(id)`	—	`closePanelOnOverlay` checks `e.target === overlay` so clicking *inside* the panel does not close it.

8.2 Player pages

`player/dashboard.html` — 17 controls

Button	Calls	Touches	Then
Featured game ◀ ▶	`setFeaturedGame(i)`	—	Changes `window._featGameId` and repaints the hero card.
Play (hero / card)	`playFast(gameId)`	`sessions`	Resolves through `GameLaunch`; creates a solo session and navigates to the game or the wrapper.
Details	`viewBiodata(gameId)`	—	Opens the biodata modal with description, genre, player count, rating.
Next match ▶	`slideNextMatch()`	—	Carousel over the player's recent `match_results`.
Friend name	`showUserProfile(name, event)`	—	Global helper (`utils.js:1441`). Reads live `Presence.statusOf()`, tier from points, wins/losses, auto-highlight image.
Games / Tournaments / Profile / Leaderboard / Match History / My Tournaments / Notifications	`navigate('<page>')`	—	Plays the 220 ms exit animation then sets `location.href`. See §5 note on `navigate` vs `navigateTo`.
Coming-soon tile	`Toast.info('Game coming soon!')`	—	Shown when `GameLaunch.resolve()` returns null.

`player/games.html` — 22 controls (the busiest player page)

Button	Calls	Touches	Then
Quick Play	`quickPlay(gameId, e)`	`sessions`, `player_session`, localStorage queue	Checks `hasEngine()`, expires stale queue entries, then either **instantly matches** a waiting player from `localStorage[QP_WAIT(gid)]` or registers this tab as the waiter and opens the searching modal.
Cancel matchmaking	`cancelMatchmaking()`	localStorage queue	Removes the waiter entry, emits `quick_play_cancel`, closes the modal.
Single Player	`launchSinglePlayer(gameId, e)`	`sessions`	Creates a solo session and launches. Used by Hill Climbing, 2048, Hextris.
Join Lobby	inline: `if(hasEngine(id)) location.href='join-session.html?gameId='+id`	—	Note the guard is written *inline in the attribute*. If no engine, shows "Lobby coming soon".
♥ Favourite	`toggleFav(id, event)`	`favorites`	Keyed by user id: `DB.favorites[uid] = [gameIds]`. Calls `event.stopPropagation()` so the card underneath does not also open.
Show favourites only	`toggleFavView()`	—	Flips a page-local flag and calls `applyFilters()`.
Game card	`viewGame(id)`	`games.views`	Opens the detail panel, increments the view counter, lists open sessions for that game and its reviews.
★★★★★ rating	`selectRating(i)` → `submitReview(gameId)`	`reviews`, `games.rating`	Stores one review per player per game and recomputes the game's average rating.
Join this room	`joinFromDetail(sid)`	`player_session`	Capacity + status checks, then navigates to the room lobby.
Back to my session	`goToMySession()`	—	The persistent banner shown while you are in a live room.
Leave session	`leaveMySession()`	`sessions`, `player_session`	Removes you from the junction table; if you were the host, the room is deleted.
Chess vs AI / vs Friend	direct `location.href='../../games/chess/index.html?mode=ai'`	—	The two builtin games bypass the wrapper and are navigated to directly — see `BUILTIN` in `game-launch.js:31`.

`player/join-session.html` — 22 controls

Button	Calls	Touches	Then
Create / Waiting tabs	`switchTab('create'|'waiting')`	—	Show/hide panels.
Create Session	`createSession()`	`sessions`, `player_session`	Validates the name (3–40 chars), refuses if you are already in a live room, builds a full session row (`session_id = Date.now() % 999999`), adds the host to the junction table, then `navigateTo('room-lobby.html?sid=…')`.
Join	`joinSession(sid)`	`player_session`	Capacity check, status check, then to the lobby.
Leave	`leaveSession(sid)`	`player_session`, `sessions`	Removes the junction row; host leaving deletes the room.
Join by code	`joinByCode()`	`player_session`	Looks up a session id typed by the user — how a friend on another laptop joins.
Spectate by code	`spectateByCode()`	—	Routes to `GameLaunch.spectate()` if the game supports a watch-only view.
Kick / Ban	`kickPlayer(pid)` / `banFromRoom(pid)`	`player_session`, session ban list	Host-only. Kick allows a rejoin; ban does not.
Start Game	`startGame()`	`sessions.status`	Sets `in-progress` and launches.
Delete session	`deleteSession()`	`sessions`	Host-only room teardown.
Refresh	`refreshAll()`	—	`silentRefresh()` + a Toast. Mostly redundant now that the live sync exists.
Back to Games	`navigateTo('games.html')`	—	**Note:** `navigateTo`, not `navigate` — it awaits `DB.whenSynced()` first.

`player/room-lobby.html` — 13 controls

Button	Calls	Touches	Then
Start Match (host)	`startGame()`	`sessions.status`, `sessions.chess.colors`	Checks the game's minimum player count, opens a confirm dialog, sets `in-progress`, assigns chess colours (host = white, second = black), `DB.save()`, then `navigateTo()`.
Send chat	`sendChat()`	`sessions[].chat`	Pushes `{senderId, sender, text, time}` into the session row itself — chat rides along with the session and syncs with it.
Invite friends	`inviteFriends()` → `sendInvite(uid, name)`	`notifications`	Creates a notification with `type:'invite'` and a `sessionId`; the recipient's drawer renders "⚔ Join Room" / "Decline" buttons for it.
Manage player	`openManagePlayer(pid, name)`	—	Host-only kick/ban panel.
Leave	`leaveSession()`	`player_session`	Exits the room.
Advance (tournament)	`advanceTournament(round, match, winnerId)`	`tournament_matches`	Reports a bracket result and feeds the winner into the parent match.

Other player pages

Page	Buttons	Behaviour
`tournaments.html`	`viewTournament`, `openRegModal`, `registerForTournament`, `unregisterForTournament`, `enterTournamentMatch`, `playHillClimbing`, `confirmTournResult`, `disputeTournResult`, `setTab`	Registration writes to `player_tournament` and bumps `tournaments.participants`, with rules R-TRN-3/4/5/6 checked client-side. The detail view has four tabs — Overview, Format, Bracket, Leaderboard — built by `buildBracket()`, `buildPrizeBreakdown()`, `buildTimeline()` and `buildHillLeaderboard()`.
`my-tournaments.html`	`switchTab('live'|'upcoming'|'archive')`, `viewTournamentDetail(id)`	Filters `player_tournament` for your id and groups by tournament status.
`match-history.html`	`setResultFilter('win'|'loss'|'draw'|'')`, `goToPage(n)`, `toggleDetails(sessionId, el)`	Client-side pagination and filtering over `match_results`; result and Carys values are derived by `_deriveResult()` / `_computeCarys()` for legacy rows.
`match-details.html`	`jumpTo('start'|'prev'|'next'|'end')`, `jumpToIndex(i)`, `history.back()`	A move-by-move replay viewer for chess matches.
`leaderboard.html`	`showUserProfile(name, event)`, `navigateTo('dashboard.html')`	Sorts `player_profiles` by `carys_points` descending; the tier badge comes from `Economy.getTierFromPoints()`.
`notifications.html`	`setTab`, `markRead`, `markAllRead`, `clearAll`, `toggleReply`, `sendReply`, `acceptFriendReq`, `rejectFriendReq`, `acceptInviteReq`, `rejectInviteReq`	The full-page twin of the drawer. Read state uses `setNotifRead(n, uid)`, which writes `readBy[]` for broadcasts and `read=true` for direct rows. "Clear All" **tombstones** with `cleared` rather than splicing, because notifications are append-only.
`profile.html`	`Modal.open('edit-modal')` + `prefillEditForm()`, `saveProfile()`, `submitReport()`, `contactModerator()`	Editing writes back to the `users` row and the `player_profiles` row. Password change goes to `POST /auth/change-password`.
`play.html`	`toggleFullscreen()`, `exitGame()`	Only two buttons — this page is the **iframe wrapper** for uploaded and self-hosted games. Its real work is the postMessage bridge (§12.4).

8.3 Publisher pages

`publisher/publish.html` — the 4-step wizard, 13 controls

Button	Calls	Touches	Then
Next / Back	`nextStep(n)` / `prevStep(n)`	—	Validates the current step before advancing. Steps: Basics → Delivery → Assets → Review.
Genre chip	`selectGenre(name)`	—	Sets `selectedGenre`; options come from `DB.genres`, which publishers manage themselves.
PlayCarys Hosted	`selectHosting('hosted')`	—	Sets `hostingMode` and reveals the `.zip` upload field.
Self-Hosted	`selectHosting('selfhosted')`	—	Reveals the game URL + optional webhook URL fields.
Choose thumbnail	`document.getElementById('f-thumbnail-file').click()` → `handleThumbnailUpload(input)`	—	Reads the file with `FileReader.readAsDataURL()` and stores the whole image as a `data:` URL in `game.img`. **This is why the API body limit is 25 MB.**
Choose .zip	`document.getElementById('f-gamefile').click()` → `handleGameZipUpload(input)`	—	Client-side pre-validation before the real server-side check.
Submit Game	`submitGame()`	`games`, `activityLog`, `notifications`	See the full flow in §9.2.
Copy API key	`copyApiKey()`	—	`navigator.clipboard.writeText()`. The key is shown for the SDK snippet.

`publisher/mygames.html` — 14 controls

Button	Calls	Touches	Then
All / Approved / Pending / Rejected	`setFilter(status)`	—	Sets page state, calls `renderGames()`.
Edit	`openEditPage(id)` → `saveEdit()`	`games`	Metadata edits are free; changing delivery config (URL or bundle) sends the listing **back to pending** under rule R-REVIEW-3.
Delete	`deleteFromEdit()`	`games`, `deleted_games`	Goes through `DB.deleteGame()` so a tombstone is written — otherwise the union in `db-import` resurrects it.
Analytics	`viewAnalytics(id)`	—	Sessions played, views, rating, royalty revenue from `publisher_profiles.revenue_pcp`.
Cancel	`cancelEdit()` → `showView('main-view')`	—	The page uses a **view-swapping** pattern: several full-page `<div>`s, one visible at a time. No navigation involved.

Other publisher pages

Page	Buttons	Behaviour
`publish-tournament.html`	`nextStep` / `prevStep`, `selectGame`, `submitTournament`, `showMgmt`, `startTournament`, `advanceRnd`, `forceWinner`, `deleteTourn`	**Start Tournament** calls `DB.generateBracket(id)` — the mathematical fold algorithm in §9.6. **Advance Round** promotes winners into parent matches. **Force winner** is the organiser's manual override for a disputed match.
`manage-genres.html`	`openAddModal`, `openEditModal(id)`, `saveGenre`, `deleteGenre(id)`	CRUD over `DB.genres`. This array feeds the genre chips on the publish wizard and the filters on the player games page.
`publisher/dashboard.html`	`navigate(...)`, `dispatchLiveEvent('low_gravity'|'heavy_gravity')`	The live-event buttons push a row into `DB.live_events` — a demonstration of publisher-driven runtime modifiers.
`publisher/profile.html`	`Modal.open('edit-modal')`, `prefillEditForm`, `saveProfile`	Studio name, founded date, contact details.

8.4 Moderator pages

Button	Calls	Touches	Then
`moderator/reports.html` **— 15 controls**			
Review	`reviewReport(id)`	—	Read-only detail view.
Take Action	`takeAction(id)`	`reports.status`	Flips `pending → under-review` immediately (so two moderators do not both work the same report), saves, then renders the action panel.
Severity Low/Med/High/Critical	`setSeverity(val, btn)`	—	Sets a page variable and highlights the chosen chip.
Warn Player	`warnPlayer(id)` → `submitWarn()`	`moderator_actions`, `notifications`, `reports`	Appends a `warning` action, sends the player a direct notification, marks the report resolved (R-MOD-5).
Resolve	`resolveReport()`	`reports`, `moderator_actions`	Writes an `actionNote` and sets status `resolved`.
Dismiss	`dismissReport()`	`reports.status`	Sets `rejected`. R-RPT-6: cannot be re-opened afterwards.
Escalate to Ban	`escalateToBan()`	`bans`, `users.status`, `player_profiles.player_status`	The heavy one. R-MOD-4 requires **all three** writes: a ban row, `user.status='banned'`, and `player_status=false`.
`moderator/bans.html` **— 5 controls**			
Ban a player	`showBanView(userId)` → `issueBan()`	`bans`, `users`, `player_profiles`	Level 1–10, a duration and a reason. R-MOD-3 blocks a second simultaneous ban.
Unban	`unbanPlayer(userId)`	`bans.active`, `users.status`	Sets `active=false` and restores `status='active'`. The ban row is **kept** as history.
Back	`showMain()`	—	View swap.
`moderator/dashboard.html` & `sessions.html`			
Contact Admin	`contactAdmin()` → `submitContactAdmin()`	`notifications`	Direct message to an admin with a priority chip set by `setPriority(level, btn)`.
Monitor session	`monitorSession(sid)`	—	Read-only live view of a room: players, mode, duration, chat volume.

8.5 Admin pages

Button	Calls	Touches	Then
`admin/hub.html` **— 37 controls, the densest page in the project**			
Section tabs ×7	`showSection('overview'|'users'|'games'|'tournaments'|'analytics'|'audit'|'system')`	—	Single-page app inside one file. Each section has its own `render*()` function.
Add / Edit / View user	`openAddUserModal()`, `editUserFromHub(id)`, `viewUserDetails(id)`, `saveUser()`	`users`	Full user CRUD from the hub.
Quick ban / unban / delete	`quickBan(id)`, `quickUnban(id)`, `quickDelete(id)`	`users`, `bans`	One-click moderation without leaving the table.
Toggle / delete game	`quickToggle(id)`, `quickDeleteGame(id)`, `openGameReview(id)`	`games`, `deleted_games`	Enable/disable respects R-GAME-4. Delete writes a tombstone.
Create / Start / End / Delete tournament	`createTournament()`, `startTournament(id)`, `endTournament(id)`, `deleteTournament(id)`	`tournaments`, `tournament_matches`, `deleted_tournaments`	Admin-side twin of the publisher tournament tools.
Export Users / Games / Reports / Audit	`exportData(type)`	—	Builds a CSV string in memory, creates a Blob, and triggers a download with a temporary `<a download>`.
Clear audit log	`clearAuditLog()`	`activityLog`	Empties the array after a confirm dialog.
Danger zone ×4	`dangerAction('purge-sessions'|'reset-bans'|'clear-reports'|'reset-platform')`	`sessions`, `bans`, `reports`, `activityLog`	All four route through one function with a `{title, msg, fn}` lookup table, wrapped in `confirmAction()`, and each writes an audit row via `addAudit()`. **Study this function — it is the cleanest pattern in the codebase.**
`admin/users.html` **— 15 controls**			
Warn	`warnUser(id)`	`notifications`	Sends a moderation notice only — no status change.
Suspend / Revoke suspension	`suspendUser(id)` / `revokeSuspendUser(id)`	`users.status`	`suspended` blocks staff logins but still allows players in — see the check in `doPanelLogin`.
Ban / Unban	`banUser(id)` / `unbanUser(id)`	`users`, `bans`	Standard ban pair.
Ban & Delete	`banAndDeleteUser(id)`	`users`, `player_profiles`	Ban then remove the account and its profile row (R-USR-5).
`admin/approvals.html`, `game-review.html`, `settings.html`			
Open review	`openReview(gameId)`	—	The approvals list is deliberately one button per row — a queue, not a control panel.
Load preview	`loadPreview()`	—	Embeds the submitted build in an iframe so

			the reviewer can actually play it before deciding.
Re-scan	`rescan()`	`games.scan`	`POST /games/:id/rescan` — re-runs the static scanner on the installed build.
Approve / Reject	`decide('approved'|'rejected')`	`games`, `game_reviews`, `publisher_profiles`	`PATCH /games/:id/approve`. Full flow in §9.3.
Save / Reset settings	`saveSettings()`, `resetDefaults()`	`settings`	Platform toggles.

8.6 Superuser pages

Button	Calls	Touches	Then
`superuser/admins.html` **— the only place roles change**			
Create Admin	`openCreateModal()` → `createAdmin()`	`users`	Creates a `role_id: 2` account. Registration cannot do this (R-AUTH-4), so this page is the sole entry point.
Promote to Admin	`promoteToAdmin(id)`	`users.role_id`	3 → 2. Server-side twin is `PATCH /users/:id/role`, guarded with `@Roles('superuser')`.
Demote to Moderator	`demoteToModerator(id)`	`users.role_id`	2 → 3. R-USR-2 forbids demoting the superuser itself.
Suspend / Reinstate	`suspendUser(id)` / `reinstateUser(id)`	`users.status`	Staff account control.
`superuser/users.html`, `games.html`, `reports.html`			
Add / Edit user	`openAddModal()`, `addUser()`, `openEditModal(id)`, `saveEdit()`	`users`, profiles	Any role can be created here, including staff.
Quick suspend / reinstate / ban / unban	`quickSuspend`, `quickReinstate`, `quickBan`, `quickUnban`	`users`, `bans`	Inline table actions.
Toggle / Delete game	`toggleGame(id)`, `deleteGame(id)`, `openGameReview(id)`	`games`, `deleted_games`	Same rules as the admin hub.
Resolve / Dismiss report	`openResolveModal(id, action)` → `submitResolve(action)`	`reports`, `moderator_actions`	Superuser override of moderator decisions.
Re-open report	`reopenReport(id)`	`reports.status`	The one place R-RPT-6 can be overridden.
Issue / Lift ban	`openBanModal(userId)` → `issueBan()`, `liftBan(id)`	`bans`, `users`	Platform-wide ban control.
`superuser/settings.html` **— 14 toggles + 5 exports + 4 danger actions**			
Toggles ×14	`makeToggle(el, initial, onChange)`	`settings`	Maintenance mode, open registration, 2FA, strong passwords, email/report/approval alerts persist to `DB.settings`. The other six are UI-only demonstrations that just Toast — worth knowing before a viva question.
Export CSV ×5	`exportCSV('users'|'games'|'reports'|'bans'|'tournaments')`	—	Blob + download link, same technique as the admin hub.
Danger zone ×4	`dangerAction('purge-sessions'|'lift-bans'|'archive-reports'|'reset-data')`	`sessions`, `bans`, `reports`, `activityLog`	Same lookup-table pattern as the hub.
Save platform info	`savePlatformInfo()`	`settings`	Platform name, session timeout.

8.7 Global buttons — present on every signed-in page

These are injected by `buildNavbar()` and `injectSideDrawer()` in `js/utils.js`, so they exist on all 35 role pages without any page knowing about them.

Button	Calls	Behaviour
☰ Hamburger (+ red badge)	`toggleDrawer()`	Opens the right-hand drawer. The badge count comes from `notifIsVisible(n, uid, role) && !notifIsRead(n, uid)`.
Logo / brand	`Auth.gotoDashboard()`	Role-aware home.
Nav links	`navigate('<page>')`	Link sets differ per role — the `navLinks` object at `utils.js:1149`.
Avatar	`showUserProfile(username, event)`	Your own profile card.
⬅️ Logout	`Auth.logout()`	Token drop → guest cleanup → claim release → `flushSync` → redirect.
Drawer: + Add friend	`toggleAddFriendUI()` → `submitAddFriend()`	Exact-username lookup, four guards (not found / self / not a player / already friends / already requested), then pushes a `type:'friend_request'` notification.
Chat with friend	`toggleFriendChat(name)` → `sendInlineMsg(name)`	Collapsible per-friend chat box that sends a direct notification.
Remove friend	`removeFriend(name)`	Confirm dialog, then filters `user.friends`.
Accept / Reject request	`acceptFriendReq(id)` / `rejectFriendReq(id)`	Accept writes a friend entry into **both** users' `friends` arrays and rewrites the notification into a plain system row so the buttons disappear.
✓ Read / Mark all read	`markNotifRead(id)` / `markAllNotifRead()`	Uses `setNotifRead()`, then `_smoothRemoveNotif()` animates the item out instead of repainting the whole drawer.
Reply	`sendNotifReply(id)`	Enforces the role-based reply matrix (players↔players, publishers→admins only, moderators not to superusers/publishers) and **refuses to reply into a broadcast row**, because that row is shared by the whole role.
Send Message (staff)	`sendDirectMessage()`	Same permission matrix. Hidden entirely for guests, players and publishers.
Publisher support panel	`sendPublisherMessage()`	Subject dropdown + 500-char body → picks a random admin → direct notification with a `publisher_support` type and a conversation history view.
Device handoff banner	Confirm / Cancel	Appears only when another device asks to sign in as you (§6.6). Auto-denies after 15 s.

Exercise — trace one button of each kind Pick one **read-only** button (`viewGame`), one **simple mutation** (`toggleFav`), one **multi-table mutation** (`escalateToBan`) and one **API-calling** button (`decide('approved')`). For each, write down: which arrays change, whether `DB.save()` is called, and what the *other* laptop sees. If you can do that for these four, you can do it for all 400.

PART 9

End-to-end flows

Eight complete journeys, from the click to the bytes on disk. These are the answers to "explain how X works in your project".

9.1 Sign in

```
USER types credentials, clicks Sign In
   │
 ① doPanelLogin(uId, pId, expectedRole)            index.html:1428
   ├─ DB.users.find(...)                           local lookup by username / phone
   ├─ status checks: banned? suspended staff?
   └─ role check: userRole === expectedRole?       ← per-column strictness
   │
 ② verifyPasswordWithServer(user, password)        index.html:1394
   └─ POST /api/v1/auth/login  { email|username, password, role_id }
        │
        ▼  BACK END — auth.controller.ts → auth.service.ts:29
        ├─ findAllByEmail() → candidates            (email is unique PER ROLE)
        ├─ candidates.find(u => u.password === dto.password)
        ├─ if (user.status === 'banned') → 401
        ├─ updateLastSeen(user_id)
        └─ return { …user…, token: signToken(user, roleName) }
             └─ HMAC-SHA256(base64url(payload), SECRET)   token.util.ts:92
        ▼
   └─ Auth.setApiToken(token)   → sessionStorage + localStorage
   │
 ③ Auth.signInWithHandoff(user)                    utils.js:530
   ├─ account free?      → confirm dialog → forceClaimSession()
   └─ account claimed?   → socket 'device_login_request' → old device approves/denies
   │
 ④ Auth.login(user, token)  → writes 'playcarys_session' to BOTH storages
 ⑤ DB.save()                → POST /db-import so the claim reaches the server  ★ do not remove
 ⑥ await DB.whenSynced(1500)
 ⑦ window.location.href = rolePages[user.role]
   │
   ▼ DASHBOARD LOADS
     ├─ db.js hydrates from localStorage, then GET /db-export, then mergeRemoteState()
     ├─ Auth.requireAuth(['player'])   → session found, role allowed
     ├─ buildNavbar('player','dashboard')
     │    ├─ auto-marks moderator/system notifications read
     │    ├─ computes the unread badge
     │    ├─ injectSideDrawer(session)
     │    └─ Presence.start()   → beat() now, then every 45 s
     └─ page render()
```

9.2 Publish a game (hosted model)

```
PUBLISHER fills the 4-step wizard, clicks Submit Game
   │
 ① submitGame()                                  publish.html
   ├─ duplicate-title check across DB.games
   ├─ build newGame with game_id = max(existing)+1
   │    · BOTH naming styles written on purpose:
   │      hostingType + hosting_type, gameUrl + game_url
   │      (existing pages read camelCase; the store uses snake_case)
   │    · game_approval_status: 'pending'   game_status: 'disabled'   ← R-GAME-2
   ├─ DB.games.push(newGame)
   ├─ DB.activityLog.unshift({type:'approval', …})
   ├─ dispatchRoleNotification('admin',  …)   one DIRECT row per admin
   ├─ dispatchRoleNotification('superuser', …)
   └─ await DB.save()        ★ must land BEFORE the upload — /games/:id/bundle
   │                           rejects an id the server has never seen
 ② uploadBundle(gameId, publisherId)
   ├─ FormData: bundle=<.zip>, publisher_id
   └─ POST /api/v1/games/:id/bundle    (multipart, no Content-Type header set by us)
        │
        ▼  BACK END — game.controller.ts:109 → game.service.installBundle()
        ├─ FileInterceptor limits: 50 MB, 1 file
        ├─ ownership check (R-GAME-3): publisher_id must own the game
        ├─ BundleService.install(buffer, name, id)          bundle.service.ts:114
        │     ├─ size cap 50 MB
        │     ├─ readZip() — reject zip-slip paths, zip bombs
        │     ├─ reject .exe .bat .sh .dll .php .py … and .htaccess / web.config
        │     ├─ require index.html at the root
        │     ├─ slug = "neon-braker-20"  (title + game_id, so two studios can both ship "Chess")
        │     ├─ extract to front-end/games/<slug>/
        │     └─ write games/.bundles.json sidecar   ← survives a db wipe
        ├─ ScanService.scan(dir)                            scan.service.ts
        │     ├─ hash every file (sha256) → bundle_hash
        │     ├─ regex rules per file → findings[]
        │     └─ verdict: pass | warn | fail
        └─ game.play_path = "games/neon-braker-20";  game.scan = report
   │
 ③ back on the client: newGame.play_path = uploaded.play_path;  DB.save();
 ④ navigate('mygames')  → listing shows "Pending review"
```

Why the sidecar file `games/.bundles.json` exists An uploaded build is a *filesystem* fact, but `play_path` lived only in memory and in `live_database.json` — which any client's `db-import` rewrites wholesale. Both can lose it, which **orphans the extracted files**: the build sits on disk while the platform has forgotten it exists. Keeping the mapping next to the files it describes means the two can never disagree.

9.3 Admin review and approve

```
ADMIN opens approvals.html → openReview(gameId) → game-review.html
   │
 ① load()  → GET /api/v1/games/:id/review        @Roles('admin','superuser')
   returns: the game, the scan report, the file manifest, the review history
   ├─ verdict badge  pass / warn / fail
   ├─ findings list: code, severity, file, line, snippet
   └─ external_origins: every third-party host the build talks to
 ② loadPreview()   → iframe of the actual build, so the reviewer can PLAY it
 ③ rescan()        → POST /games/:id/rescan      re-runs the scanner
 ④ decide('approved')  →  PATCH /api/v1/games/:id/approve
        │
        ▼  game.service.approveGame()
        ├─ R-REVIEW-1  a hosted game with NO scan cannot be approved at all
        ├─ R-REVIEW-4  verdict === 'fail'  → only a SUPERUSER may override
        ├─ R-GAME-5    game_status = 'enabled'      (auto-enable on approval)
        ├─ R-REVIEW-2  approved_bundle_hash = scan.bundle_hash   ← binds the
        │                decision to THIS EXACT BUILD, not to the listing
        ├─ R-REVIEW-5  append an immutable row to game_reviews:
        │                { reviewer_id, decision, bundle_hash, scan_verdict,
        │                  scan_summary, note, override, decided_at }
        └─ R-GAME-5    publisher_profiles.approved_games++ / active_games++
   │
   ▼ persistence.interceptor fires → scheduleSave() → live_database.json
   ▼ the publisher's next sync shows "Approved"; the game appears in /games?active=true
```

The single best idea in this module: approval binds to the BUILD `game_approval_status` says whether a **listing** is approved. `approved_bundle_hash` says which **build** was approved. Without the second field, a publisher could pass review with a clean bundle and then re-

upload anything they liked while the listing stayed live. Now any drift between `bundle_hash` and `approved_bundle_hash` means **re-review required** (R-REVIEW-3).

9.4 Create room → play → record result

```
① join-session.html → createSession()
     DB.sessions.push({ session_id, game_id, created_by, host, hostId,
                        maxPlayers, status:'waiting', mode, privacy,
                        players:[uid], start_time, end_time })
     DB.player_session.push({ player_id: uid, session_id })
     DB.save()  →  navigateTo('room-lobby.html?sid=…')      ← awaits whenSynced

② OTHER PLAYER joins  →  joinSession(sid)  →  player_session row  →  DB.save()
     └ socket 'global_db_update' → the host's PlayCarysLive.pull()
       → mergeRemoteState() → synthetic 'storage' event → lobby re-renders
       → the host SEES the new player without refreshing        ★ this is the fix
                                                                  for "doesn't work
                                                                  without refresh"

③ HOST clicks Start Match  →  startGame()
     ├ minimum-player check from game.players ("2-8" → min 2)
     ├ confirmAction() dialog
     ├ currentSession.status = 'in-progress'
     ├ chess only: colors = { hostId:'w', secondId:'b' }
     ├ DB.save()
     └ navigateTo(chess ? '…/games/chess/index.html?sid&color'
                        : GameLaunch.launch(gid, {sid}))

④ THE GAME RUNS
     builtin  → the game file itself writes results through DB.recordMatchResult()
     uploaded → play.html wrapper ← postMessage ← PlayCarys.submitResult()

⑤ RESULT RECORDED  →  DB.recordMatchResult(sid, pid, gid, result, score, extra)
     ├ duplicate guard on (session_id, player_id)         ← the composite PK
     ├ default formula: win 100 / draw 25 / loss 15,  ×1.5 chess, ×1.1 hill
     ├ extraData may OVERRIDE carys_earned and xp_gained  ← games define their rules
     ├ but GAME_PRIVATE_FIELDS ['coins','diamonds','elo_delta'] are DROPPED
     ├ match_results.unshift(row)
     └ DB.awardPoints(playerId, pcp)  →  player_profiles.carys_points += pcp

⑥ Economy.awardPostMatchProgression()  (utils.js:901)
     ├ xp, level-up at floor(1000 × level^1.5)
     └ PUBLISHER ROYALTY: publisher_profiles.revenue_pcp += 15 per session
```

Why a game may not write `coins` or `diamonds` into `match_results` The platform's ledger is PlayCarys Points. Hill Climbing's coins and diamonds are that *game's* internal currency and belong in the game's own store; `elo_delta` was a chess rating the platform had no use for (Hill Climbing was passing a meaningless `0` on every run just to fill the column). Anything a game sends under those names is dropped at the boundary rather than silently entering the platform's schema. **This is API-boundary discipline and it is a genuinely strong answer in a viva.**

9.5 Quick Play matchmaking

Two implementations exist — a same-browser one using localStorage, and a cross-network one over Socket.IO.

```
SAME BROWSER (two tabs) — games.html:401
  Tab A: quickPlay(gid) → no waiter in localStorage[QP_WAIT(gid)]
                        → write { tid, uid, ts } and show the searching modal
  Tab B: quickPlay(gid) → reads the waiter, not stale (TTL), not self (different tid)
                        → INSTANT MATCH → _buildQPSession(gid, hostUid, guestUid)
                        → both tabs navigate into the game

ACROSS THE NETWORK — game.gateway.ts:176
  Client → emit 'quick_play_join'  { gameId, uid }
  Server   matchQueues: Map<gameId, [{uid, clientId}]>
      ├ remove any existing entry for this uid/socket (double-click guard)
      ├ look for an opponent with a DIFFERENT uid and clientId
      ├ found → splice from queue
      │          isHost = client.id > opponent.clientId    ← deterministic tie-break,
      │                                                      avoids both sides thinking
      │                                                      they are the host
      │          emit 'quick_play_match_found' {role:'host'|'guest', oppId, gameId}
      └ not found → push into the queue
  handleDisconnect() removes the socket from EVERY queue
```

9.6 Tournament: create → bracket → advance

`DB.generateBracket()` at `data/db.js:1139` is the most algorithmic function in the project.

```
DB.generateBracket = function (tournament_id) {
  const t = this.tournaments.find(x => String(x.tournament_id) === String(tournament_id));
  if (!t) throw new Error('Tournament not found.');   // throw, don't return quietly — see note

  // 1. Registered players, SEEDED by points (seed 1 = highest)
  const pids = this.player_tournament
    .filter(pt => String(pt.tournament_id) === String(tournament_id))
    .map(pt => String(pt.player_id))
    .sort((a,b) => (userB?.carys_points||0) - (userA?.carys_points||0));

  if (pids.length < 2) throw new Error('At least 2 players are required.');

  // 2. Round up to the next power of two: 6 players → 8 slots
  const nextPow2 = Math.pow(2, Math.ceil(Math.log2(pids.length)));

  // 3. THE FOLD ALGORITHM — produces 1v8, 4v5, 2v7, 3v6
  //    so the top seeds can only meet in the final.
  let order = [1];
  for (let i = 1; i <= Math.log2(nextPow2); i++) {
    const nextOrder = [], sum = Math.pow(2, i) + 1;
    for (let j = 0; j < order.length; j++) nextOrder.push(order[j], sum - order[j]);
    order = nextOrder;
  }
  //   i=1: [1,2]   i=2: [1,4,2,3]   i=3: [1,8,4,5,2,7,3,6]

  // 4. Slots beyond the player count become BYEs
  const seeds = order.map(slot => slot <= pids.length ? pids[slot-1] : "BYE");

  // 5. Round 1 matches, then empty shells for every later round,
  //    each carrying parentMatchId so a winner knows where to go next.
  // 6. Auto-advance every BYE.
  t.status = 'in-progress';
  this.save();
};
```

Two real bugs recorded in this function **1.** It used to `return` silently when the tournament was not found. The caller reports "Tournament Started! Bracket generated." on anything that is not an exception — so the publisher was told it worked while nothing had been created. **Now it throws.** **2.** The BYE auto-advance used `===` on the tournament id. `startTournament('3')` passes a *string* from an inline `onclick`, while seeded rows store a *number* — so it matched nothing, byes were stranded in round 1, and the bracket could never advance because those matches had no winner to feed forward. **Now** `String(a) === String(b)`**.** This is the single most common bug class in the project.

9.7 Report → moderator action → ban

```
① PLAYER: profile.html → submitReport()
     reports.push({ report_id, reported_by, reported_user, type, reason,
                    status:'pending', report_date, session, sessionId })
     ├─ R-RPT-1 cannot report yourself      ├─ R-RPT-4 reason required
     ├─ R-RPT-2 both users must exist       ├─ R-RPT-7 type from a fixed list
     └─ R-RPT-3 duplicate open report from the same reporter is rejected

② MODERATOR: reports.html → takeAction(id)
     status: pending → under-review, DB.save() IMMEDIATELY
     └─ so a second moderator opening the queue sees it is already claimed

③ Choose an outcome
   ├─ warnPlayer → submitWarn()
   │     moderator_actions.push({action_type:'warning', action_reason})
   │     notifications.push(direct row to the player)
   │     reports[i].status = 'resolved'                      ← R-MOD-5
   ├─ dismissReport()  → status 'rejected'                   ← R-RPT-6 final
   └─ escalateToBan()
         bans.push({ playerId, playerName, level, duration, reason,
                     bannedBy, bannedById, bannedDate, expiresDate, active:true })
         users[i].status = 'banned'                          ┐
         player_profiles[i].player_status = false            ┘ R-MOD-4: ALL THREE

④ EFFECTS, immediately and everywhere
   ├─ login  : auth.service throws 'Account is banned'        R-AUTH-3
   ├─ presence: statusOf() returns Offline for banned/suspended
   ├─ sessions: player_status=false blocks joining            R-SES-3
   └─ tournaments: player_status=false blocks registration    R-TRN-6

⑤ LIFT: bans.html → unbanPlayer()
     ban.active = false   (the row is KEPT as history)
     users[i].status = 'active'
```

Why a ban writes three places instead of one Each field answers a different question. `bans` is the **audit trail** — who, why, when, for how long. `users.status` is the **login gate**. `player_profiles.player_status` is the **gameplay gate** read by the session and tournament rules. Set only one and the ban half-works, which is worse than not banning at all.

9.8 Notifications and friends

Notifications are one table doing five jobs: DMs, friend requests, room invites, system messages and role broadcasts. The distinction that matters is **direct vs broadcast**.

```
// data/db.js:2514 — four tiny functions that govern the entire system
function isBroadcastNotif(n) { return (n.toUser == null) && n.toRole != null; }

function notifIsForUser(n, uid, role) {
  return (n.toUser != null) ? _notifIdEq(n.toUser, uid)     // direct → that user ONLY
                            : (n.toRole === role);           // broadcast → whole role
}

function notifIsRead(n, uid) {
  if (!isBroadcastNotif(n)) return !!n.read;                 // direct → simple flag
  return (n.readBy || []).some(x => _notifIdEq(x, uid));     // broadcast → per-user list
}

function setNotifRead(n, uid) {
  if (!isBroadcastNotif(n)) { n.read = true; return; }
  if (!Array.isArray(n.readBy)) n.readBy = [];
  if (uid != null && !n.readBy.some(x => _notifIdEq(x, uid))) n.readBy.push(String(uid));
}
```

Three real notification bugs, all from one root cause **1. Inclusive OR.** The visibility test used to be `n.toRole === role || n.toUser === uid`. `dispatchRoleNotification()` set **both** fields on each per-recipient copy — so with 2 admins each admin saw the message *twice*, and any per-user message written that way reached the entire role. The test is now **exclusive**: a row addressed to a user is not also delivered to their role.

2. Shared read state. A broadcast is **one row** read by everyone in the role. Pressing "✓ Read" flipped `n.read` and cleared the announcement *for every other moderator*. Fixed with `readBy[]`, and the merge unions that array rather than overwriting it.

3. Seed rows mis-addressed. Rows 5, 6, 7 and 10 were personal messages ("your game has been approved") addressed to a whole role — so every

> publisher was told a game they do not own had been approved. `DB.repairNotificationAddressing()` fixes them on **every merge**, not once behind a flag, because the server keeps re-sending the unrepaired rows.

The role-based messaging matrix

Sender	May message / reply to
Player	Other players only.
Publisher	Admins only (via the dedicated support panel with subject categories).
Moderator	Admins, moderators and players — **not** superusers or publishers.
Admin / Superuser	Anyone.

Nobody may reply into a broadcast row — the reply would be appended to the row the whole role is reading.

PART 10

NestJS back-end anatomy

Twelve feature modules, all built from the same five files. Learn the pattern once and every module reads the same way.

10.1 The 5-file module pattern

Open any folder under `back-end/src/` — `report/` is a good small example — and you find the same five files:

File	Decorator	Responsibility
`*.module.ts`	`@Module`	Wiring only. Declares which controller and which providers belong together.
`*.controller.ts`	`@Controller`, `@Get`, `@Post` …	**HTTP only.** Maps a URL to a method, extracts params/body, calls the service. No business logic.
`*.service.ts`	`@Injectable`	**All the rules.** Every `R-XXX-n` lives here. Throws `NotFoundException`, `BadRequestException`, `ForbiddenException`, `ConflictException`.
`*.repository.ts`	`@Injectable`	**Data access only.** `find`, `filter`, `push` against the arrays in `store.ts`. If a real database ever replaced the arrays, **only this file would change.**
`*.dto.ts`	`@IsString`, `@IsInt` …	The shape of a request body, plus validation rules and Swagger docs.

The pattern in one screen

```
// report.module.ts — wiring
@Module({
  controllers: [ReportController],
  providers:   [ReportService, ReportRepository],
  exports:     [ReportService],
})
export class ReportModule {}

// report.controller.ts — HTTP only
@Controller('reports')                      // → /api/v1/reports
export class ReportController {
  constructor(private readonly reportService: ReportService) {}   // ← DI, see below

  @Post()
  create(@Body() dto: CreateReportDto) {          // dto already validated by the global pipe
    return this.reportService.create(dto);        // no logic here — just delegate
  }
}

// report.service.ts — the rules
@Injectable()
export class ReportService {
  constructor(private readonly repo: ReportRepository) {}

  create(dto: CreateReportDto) {
    const reporter = users.find(u => u.user_id === dto.reported_by);
    if (!reporter) throw new NotFoundException(`Reporter ${dto.reported_by} not found`);  // R-RPT-2
    if (dto.reported_by === dto.reported_user)
      throw new BadRequestException('A player cannot report themselves');                 // R-RPT-1
    // …R-RPT-3, R-RPT-4, R-RPT-7…
    return this.repo.create({ …dto, status: 'pending', report_date: today });
  }
}
```

Dependency injection, explained without jargon You never write `new ReportService()` anywhere. You *declare* in the constructor that you need one, and Nest builds the object graph at startup — reading the `providers` arrays to know what can be injected where.

Practical benefits: one shared instance instead of many; swap a real database in by changing only the repository; and in tests you can inject a fake. The magic is `reflect-metadata` plus `emitDecoratorMetadata` in `tsconfig.json` — which is why that flag must stay on.

Why the extra repository layer, when the "database" is just an array It looks like ceremony for `users.find(...)`. The value is that **the service never knows how data is stored.** Migrating this project to PostgreSQL would mean rewriting twelve repository files and nothing else — no controller, no rule, no DTO would change. That is exactly the argument to give if asked "how would you scale this?".

10.2 Bootstrap — `main.ts`, in order

```
const app = await NestFactory.create(AppModule);

// ① Body limit: 25 MB, because a publisher's cover image is stored inline as a data: URL
app.use(express.json({ limit: '25mb' }));   // Express default is 100 KB → 413 errors

// ② Global validation for every DTO
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,               // strip properties with no decorator
  forbidNonWhitelisted: false,   // …but don't reject the request for having them
  transform: true,               // turn the plain body into the DTO class
  transformOptions: { enableImplicitConversion: true },  // "5" → 5 for an @IsInt field
}));

// ③ CORS — the site is on :8080, the API on :3000, so they are different ORIGINS
app.enableCors({ origin: '*',
                 methods: ['GET','POST','PATCH','DELETE','OPTIONS'],
                 allowedHeaders: ['Content-Type','Authorization','role'] });

// ④ One guard for every route
app.useGlobalGuards(new RolesGuard(app.get(Reflector)));

// ⑤ One interceptor that persists after every successful write
app.useGlobalInterceptors(new PersistenceInterceptor());
installShutdownFlush();

// ⑥ Every route gets the /api/v1 prefix
app.setGlobalPrefix('api/v1');

// ⑦ Swagger UI at /api/docs + a swagger.json dump to back-end/docs/
await app.listen(port, '0.0.0.0');   // 0.0.0.0, not 127.0.0.1 → other laptops can reach it
```

Two lines that people get wrong and then debug for an hour `'0.0.0.0'` — bind to all interfaces. With the default `localhost` the API is invisible to every other machine, and the second laptop sees "cannot reach the server".
`allowedHeaders` — if `Authorization` is missing from this list, the browser's CORS preflight blocks every authenticated request and you get a confusing CORS error rather than a 401.

The request pipeline, in execution order

```
HTTP request
   ↓
[ Middleware ]     express.json (25 MB), CORS
   ↓
[ Guards ]         RolesGuard.canActivate()  →  403 if it returns false / throws
   ↓
[ Pipes ]          ValidationPipe (DTO), ParseIntPipe (:id)  →  400 on bad input
   ↓
[ Controller ]     the handler method
   ↓
[ Service ]        domain rules  →  404 / 400 / 403 / 409
   ↓
[ Repository ]     array access
   ↓
[ Interceptor ]    PersistenceInterceptor.tap()  →  scheduleSave()   ← ONLY on success
   ↓
HTTP response
```

10.3 DTOs — validation as declaration

```
export class LoginDto {
  @ApiProperty({ description: 'The email address', required: false })
  @IsOptional() @IsEmail()
  email?: string;

  @ApiProperty({ description: 'Alternatively, the username', required: false })
  @IsOptional() @IsString()
  username?: string;

  @ApiProperty({ description: 'The password' })
  @IsString() @MinLength(8)
  password: string;

  @ApiProperty({ description: '4 = Player, 5 = Publisher. Omit and the password decides.', required: false })
  @IsOptional() @IsInt()
  role_id?: number;
}
```

Each decorator does three jobs at once:

Decorator	Effect
`@IsEmail()`	Rejects a malformed address with a **400** before the controller runs.
`@ApiProperty()`	Adds the field to Swagger with its description and example.
the TypeScript type	Compile-time safety inside the service.

Try it

```
curl -X POST http://localhost:3000/api/v1/auth/login \
     -H 'Content-Type: application/json' -d '{"email":"not-an-email","password":"x"}'
```

You get a **400** listing both failures — wrong email format *and* password too short — and `AuthService.login()` is never entered. You wrote zero validation code to get that.

10.4 Persistence — one writer, debounced, atomic

The interceptor (34 lines) — why it is global

```
const MUTATING = new Set(['POST', 'PUT', 'PATCH', 'DELETE']);

intercept(context, next) {
  if (context.getType() !== 'http') return next.handle();
  if (!MUTATING.has(context.switchToHttp().getRequest()?.method)) return next.handle();

  // tap's next callback runs ONLY on success, so a rejected request
  // (validation error, 403, conflict) never triggers a write.
  return next.handle().pipe(tap(() => scheduleSave()));
}
```

Why global instead of a `save()` call in each handler The file's own comment answers it: *"there are 37 mutating endpoints across 12 modules, and any one of them forgotten would silently lose data on the next restart — the exact failure this is meant to end. One interceptor cannot be forgotten by a future endpoint."*

The writer — `persistence.ts`

```
export function saveNow() {
  if (writing) { dirtyWhileWriting = true; return; }     // re-entrancy guard
  const target = dbPath();
  const tmp    = `${target}.tmp-${process.pid}`;
  try {
    fs.writeFileSync(tmp, JSON.stringify(snapshot(), null, 2), 'utf8');
    fs.renameSync(tmp, target);        // ★ ATOMIC on the same volume
  } catch (err) { … }
  finally { writing = false; if (dirtyWhileWriting) scheduleSave(); }
}

export function scheduleSave(delayMs = 300) {           // debounce
  if (timer) clearTimeout(timer);
  timer = setTimeout(() => { timer = null; saveNow(); }, delayMs);
  if (typeof timer.unref === 'function') timer.unref();   // never hold the process open
}
```

Technique	Why
Write temp, then rename	`rename` is atomic on one volume, so a crash mid-write can never leave a half-written `live_database.json`. This file is the **only** copy of the data — a torn write is unrecoverable.
300 ms debounce	A burst of mutations (a bracket advancing, a `db-import` touching 21 tables) collapses into one file write instead of dozens.
`timer.unref()`	A pending timer would otherwise keep the Node process alive after Ctrl+C.
`installShutdownFlush()`	Catches SIGINT / SIGTERM / `beforeExit` and flushes, so Ctrl+C between the debounce and the write does not discard the last few seconds.

Passthrough — the six tables the back-end does not model

```
let passthrough = {};   // settings, activityLog, genres, live_events, quickPlayQueue, telemetry

export function rememberPassthrough(body) {
  const owned = new Set(Object.keys(storeTables()));
  for (const key of Object.keys(body)) if (!owned.has(key)) passthrough[key] = body[key];
}
export function snapshot() { return { ...passthrough, ...storeTables() }; }
```

What this prevents Those six tables belong to the front-end's `db.js` and reach the file only by riding along on a `db-import` payload. Persisting only the store would **quietly delete all six** — including `settings`, which 39 pages read. Every API-driven save would wipe the platform configuration.

10.5 Swagger — free documentation

```
const config = new DocumentBuilder()
  .setTitle('PlayCarys OMGMP API')
  .setVersion('1.0')
  .addApiKey({ type:'apiKey', name:'role', in:'header' }, 'role')   // lets you set the header in the UI
  .build();
SwaggerModule.setup('api/docs', app, SwaggerModule.createDocument(app, config));
fs.writeFileSync(path.join(docsDir, 'swagger.json'), JSON.stringify(document, null, 2));
```

Try it — the fastest way to explore your own API Open **http://localhost:3000/api/docs**. Every route, every DTO field, every example is there, generated from the decorators you already wrote. You can execute requests directly from the page. This is the single best tool for learning your own back-end.

PART 11

The real-time layer

How one browser's click reaches another browser without anybody pressing refresh — and the fallback for when the socket never connects.

11.1 The gateway and its events

`back-end/src/game-session/game.gateway.ts`, 254 lines. It keeps two in-memory maps:

```
private matchQueues:       Map<string, {uid, clientId}[]>;  // gameId → players waiting
private identityToSockets: Map<string, Set<string>>;        // userId → live socket ids
```

Event (client → server)	Server emits	Purpose
`register_identity`	—	"This socket is signed in as user 7." Re-sent on **every** reconnect, because a fresh socket has a fresh id.
`device_login_request`	`device_login_incoming`	New device asks the old one for permission.
`device_login_ack`	`device_login_ack`	Old device confirms a human can actually see the prompt.
`device_login_response`	`device_login_response`	Approved or denied.
`global_db_update`	`global_db_update_received`	**The most important one.** Broadcast to everyone *except* the sender.
`join_room` / `leave_room`	—	Socket.IO room membership for per-session broadcasts.
`session_action`	`session_updated`	Room-scoped state change.
`game_action`	`game_action_received`	In-game moves.
`chat_message`	`chat_message_received`	Lobby chat.
`quick_play_join` / `_cancel`	`quick_play_match_found`	Matchmaking (§9.5).

The zombie-socket bug A registry entry outlives its tab whenever Socket.IO has not yet processed a disconnect — a closed laptop, a killed tab, dropped Wi-Fi. That zombie made a handoff request look *deliverable*, so the new device sat through the full 15-second timeout and was told "no response", locking a user out of their own account with nobody on the other end.
Fix (`game.gateway.ts:74`): prune against `server.sockets.sockets.has(id)` before deciding, so "nobody is listening" is answered *immediately and honestly*.

11.2 PlayCarysLive and the synthetic `storage` event

`js/utils.js:15`. This object is the fix for the biggest usability complaint in the project.

"It doesn't work without a refresh" `DB.save()` emitted `global_db_update`, the gateway broadcast `global_db_update_received` — and **nothing was listening for it**. `DB.mergeRemoteState()` was only ever reached from the boot hydrate. So a friend request, a chat message, another player joining your room, or the host pressing Start were all invisible until F5.

```
async pull(reason) {
  if (this._pulling) { this._queued = true; return; }        // collapse bursts
  this._pulling = true;

  await DB.whenSynced(1500);                                  // never clobber our own in-flight write
  const remote = await (await fetch(window.PC_API + '/db-export', {cache:'no-store'})).json();
  DB.mergeRemoteState(remote);
  DB.saveLocal();

  const payload = localStorage.getItem(this.DB_KEY);
  if (payload === this._lastSeenPayload) return;              // nothing actually moved
  this._lastSeenPayload = payload;

  // ★ SYNTHESISE a storage event
  window.dispatchEvent(new StorageEvent('storage', {
    key: this.DB_KEY, newValue: payload, storageArea: localStorage
  }));

  document.dispatchEvent(new CustomEvent('playcarys:dbupdate', { detail: { reason } }));
}
```

Why fake a `storage` event — this is a genuinely clever move `room-lobby`, `join-session` and the chess board all already listened for the browser's `storage` event to pick up the other player's moves. But the browser fires that event **only between tabs of the same browser**. Two players on two laptops never exchanged a single one — which is why "Start Game" launched only for the host while the guest sat in the lobby forever.

Synthesising the event here means **every one of those existing listeners starts working across the network, unchanged**. No page had to be rewritten. That is a one-line architectural fix and it is worth being able to explain out loud.

The default subscribers

```
document.addEventListener('playcarys:dbupdate', () => {
  updateNavBadge();                 // the red notification count
  window._drawerNotifRender();      // the drawer list
  window._drawerFriendsRender();    // friends + their live presence
  Auth.checkSessionOwnership();     // ← earliest point we could learn another device took over
});
```

Because these three live in `utils.js`, every page gets live notifications and live presence **without writing a line of code**. A page only adds its own listener for its own content.

11.3 The polling fallback

```
startPolling(ms) {
  this._pollTimer = setInterval(() => {
    if (document.hidden) return;                                  // background tab: don't burn cycles
    if (window.playcarysSocket?.connected) return;                // socket already covers us
    this.pull('poll');
  }, ms || 5000);
}

// boot, at the bottom of utils.js
setTimeout(initPlayCarysWebSockets, 500);
setTimeout(() => PlayCarysLive.startPolling(5000), 1500);
```

Why polling exists at all when there is a socket The Socket.IO **client** is loaded from `cdn.socket.io` at runtime. Offline — or on a locked-down college network — that script never arrives and `window.playcarysSocket` is never created. Polling every 5 s means the app still syncs, just less instantly. Two guards keep it cheap: skip while the tab is hidden, and skip entirely whenever the socket is connected.

PART 12

The game delivery pipeline

How a game goes from a publisher's .zip to a player's screen — and the three security layers in between.

12.1 Three delivery kinds

`js/game-launch.js` is the single place that answers "can this game be played, and where does it live?"

Kind	Where the code lives	How it launches	Examples
builtin	`front-end/games/<slug>/`, shipped with the platform	Direct navigation — **not** the iframe wrapper	Chess (id 7), Hill Climbing (id 8)
hosted (Model A)	Publisher's `.zip`, unpacked by the server to `front-end/<play_path>/`	iframe inside `play.html`, same origin	HexGL (16), 2048 (18), Hextris (19), Neon Braker (20)
selfhosted (Model B)	The publisher's own server	iframe with a **sandbox** attribute, cross origin	Any `https://` build

The bug that created this file Five separate pages each carried their own copy of `const implemented = ['7','8'];`. So a publisher could submit a perfectly good game and it would still show "Coming Soon" forever, because **nothing read the game record**. Every caller now delegates to `GameLaunch.resolve()`.

```
resolve: function (gameId, params) {
  const gid  = String(gameId).replace(/^g/, '');
  const game = findGame(gid);

  if (BUILTIN[gid]) return { kind:'builtin', url: webRoot()+'games/'+slug+'/index.html'+qs,
                             embeddable:false };            // keeps its historical behaviour

  if (!game) return null;
  if (!isReviewCleared(game)) return null;                  // ★ the review gate

  const playPath = field(game, 'play_path', 'playPath');
  if (playPath) return { kind:'hosted', url: webRoot()+clean+'/index.html'+qs, embeddable:true };

  const url = field(game, 'game_url', 'gameUrl');
  if (url) {
    const parsed = new URL(url, location.href);
    // Only ever embed http(s). A javascript: or data: "URL" in a game record
    // would otherwise execute in the wrapper's own context.
    if (parsed.protocol !== 'http:' && parsed.protocol !== 'https:') return null;
    return { kind:'selfhosted', url: …, embeddable:true, origin: parsed.origin };
  }
  return null;
}
```

The review gate, and its three exemptions A build is served the moment it is unpacked, but it is only *reviewed* when an admin approves it. That gap is exactly where an unreviewed bundle would be reachable by anyone holding the game id. `isReviewCleared()` closes it in the one place every launch path passes through — with three deliberate exemptions: **admins** and **superusers** (reviewing a game means playing it) and a **publisher launching their own submission** to test it.

12.2 Bundle upload and zip safety

`back-end/src/game/bundle.service.ts` + `zip.util.ts`.

Check	Attack it stops
Size cap 50 MB	Disk exhaustion. Enforced twice: by multer's `limits.fileSize` and again in `install()`.
Path traversal ("zip slip")	An entry named `../../../etc/passwd` escaping the extraction directory.
Zip bombs	A tiny archive that expands to gigabytes.
Blocked extensions	`.exe .bat .cmd .sh .dll .so .msi .vbs .ps1 .jar .php .asp .jsp .py .rb` — none can run in a browser, so their only purpose in a "web game" is to use the platform as a file drop.
Blocked names	`.htaccess`, `web.config`, `.user.ini` — files that would let a bundle **reconfigure the static server hosting it**.
`index.html` required at root	A bundle with no entry point.
Slug = `title-gameId`	Two studios can both ship a game called "Chess"; the slug is also stable across re-uploads.

12.3 The static security scanner

`back-end/src/game/scan.service.ts`, 498 lines. Read the header comment — it states the purpose precisely:

> *"bundle.service.ts already refuses archives that are malformed as ARCHIVES. That is a structural gate, and it says nothing at all about the thing a bundle is actually made of: .html and .js, which are allowed by definition because they are the game. This is the gate for that content. It never decides on its own whether a game ships — it produces evidence an admin reads: what the build touches, where, and on which line."*

Severity means CERTAINTY, not scariness

Severity	Example rule	Reasoning
critical (fails the scan)	`PC-SESSION-READ` — matches `playcarys_session`, `playcarys_db_store`	**Nothing legitimate ever does this.** A game has no reason to touch platform credentials — it is session theft.
critical	`PC-CRYPTOMINER`, `PC-SVG-SCRIPT`, `PC-CREDENTIAL-POST`	An SVG is served as an image but this one runs code. A miner silently spends the player's CPU.
high (needs a human)	`PC-STORAGE-ACCESS` — `localStorage`	A legitimate game saves progress there. **A regex cannot tell the two apart** — so an admin has to look.
high	`PC-REMOTE-IMPORT` — `import("https://…")`	Found in a real submission. The approved build and the code that actually runs become two different things, and the third party can change theirs at any time.
high	`PC-EVAL`, `PC-DYNAMIC-SCRIPT`, `PC-SERVICE-WORKER`, `PC-PARENT-ACCESS`, `PC-PLATFORM-API`, `PC-TOP-NAVIGATION`	Whatever `eval` runs is invisible to this scan. A service worker *survives the game exiting* and can intercept later requests on this origin.
medium	`PC-EXTERNAL-RESOURCE`, `PC-NESTED-IFRAME`, `PC-ENCODED-PAYLOAD`	`atob()` is a common way to hide code from exactly this kind of scan.

Practical limits, and why they are there

Limit	Reason
`MAX_SCAN_BYTES_PER_FILE = 2 MB`	A 40 MB asset blob is not worth regexing.
`MAX_HITS_PER_RULE_PER_FILE = 3`	One bad loop should not produce 9,000 findings.
`MAX_FINDINGS = 400`	The report has to stay readable by a human.
`truncated` flag on the report	So a clean report is **never** mistaken for a complete one.

Every file is also SHA-256 hashed into a `manifest`, and the whole build into `bundle_hash` — which is what makes "approval binds to the build" (§9.3) possible.

12.4 The iframe wrapper and the SDK

Two files implement the contract: `js/playcarys-sdk.js` (what the game includes) and `pages/player/play.html` (the host).

```
GAME (inside the iframe)                  HOST (play.html)
────────────────────────                  ────────────────

PlayCarys.init(apiKey, gameId)
   └─ postMessage PLAYCARYS_INIT_REQ ──────▶ verify ev.source === frame.contentWindow
                                             verify ev.origin  === resolved.origin
                            ◀──────────────── PLAYCARYS_INIT_ACK { sessionId, gameId, player }
onPlayCarysReady({sessionId})

PlayCarys.getSessionState()
   └─ PLAYCARYS_REQ_STATE ──────────────────▶
                            ◀──────────────── PLAYCARYS_STATE { … }

PlayCarys.submitResult({outcome, score,
                        carys_earned, xp_gained})
   └─ PLAYCARYS_SUBMIT_RESULT ──────────────▶ submitResult()
                                               ├─ session_guard()
                                               ├─ whitelist outcome to win|loss|draw|completed
                                               ├─ DB.recordMatchResult(...)
                                               └─ DB.save()
```

```
// play.html — the two lines that make this safe
if (ev.source !== frame.contentWindow) return;
if (resolved.origin && ev.origin !== resolved.origin && ev.origin !== 'null') return;
```

Why both checks are required Without them, **any open tab could `postMessage` a win straight into the leaderboard.** `ev.source` proves the message came from the frame we launched; `ev.origin` proves it came from the host we resolved. (`'null'` is allowed because a sandboxed iframe reports a null origin.)

The sandbox — and the one limitation to be honest about

```
if (resolved.kind === 'selfhosted') {
  frame.setAttribute('sandbox', 'allow-scripts allow-same-origin allow-pointer-lock allow-downloads');
}
```

Third-party code gets scripts and its own origin (needed for storage and WebGL) but **no top-navigation, no popups, no form submission into our page**.

Known limitation, documented in the source **Hosted** bundles run **same-origin**, because uploaded games are expected to use DB helpers the way the builtin games do. A hosted bundle can therefore read `localStorage` — which is exactly why `PC-SESSION-READ` and `PC-STORAGE-ACCESS` are scanner rules. The real fix is serving bundles from a separate port/origin. **Knowing and stating your own system's weakness is a strength in a viva, not a weakness.**

Try it — the 30-second watchdog Set a game's `game_url` to a site that refuses embedding (e.g. `https://google.com`, which sends `X-Frame-Options`). After 30 s the wrapper shows "Game did not load" with the correct explanation. Note the fix in the source: the watchdog tracks an explicit `loaded` flag rather than a CSS class, because a game that pulls slow third-party fonts delays the load event and was being declared broken while visibly running.

PART 13

Domain rules reference

Every business rule in the system, with its code. These are stated as comments at the top of each service — grep for `R-` to find them.

Authentication — `auth.service.ts`

Rule	Statement
R-AUTH-1	Password must be at least 8 characters.
R-AUTH-2	Email is unique **per role**, not globally. One address may hold a player account and a publisher account. Username stays globally unique — it is the login identifier.
R-AUTH-3	Banned users cannot log in.
R-AUTH-4	Self-registration only for player (4) or publisher (5). Enforced again in `db-import`.
R-AUTH-5	On register, the matching profile row is auto-created.
R-AUTH-6	Changing a password requires the current one, checked server-side. Also clears every other device's session.

Users — `users.service.ts`

Rule	Statement
R-USR-1	Email unique, scoped to the role the account will hold after the edit.
R-USR-2	The superuser (role 1) account cannot be deleted or demoted.
R-USR-3	Cannot assign a non-existent `role_id` — 1–5 only.
R-USR-4	When a role changes to player/publisher, a profile row is auto-created.
R-USR-5	Removing a user also removes their profile rows.

Games and review — `game.service.ts`

Rule	Statement
R-GAME-1	`game_name` is unique across the platform.
R-GAME-2	A new game starts `pending` + `disabled`.
R-GAME-3	Only the owning publisher may update or change the state of their game.
R-GAME-4	A game may only be enabled if it is already approved. **Enforced twice** — in the service and again at the `db-import` sync boundary.
R-GAME-5	Approving auto-increments the publisher's `approved_games` and auto-enables the listing.
R-GAME-6	Only approved games appear in the active list for players.
R-GAME-7	A game with active sessions cannot be deleted.
R-GAME-8	A game is only launchable once it declares delivery (a bundle or a URL).
R-REVIEW-1	Every uploaded bundle is scanned on arrival. A hosted game with no scan cannot be approved.
R-REVIEW-2	Approval binds to the **build**, not the listing — via `approved_bundle_hash`.
R-REVIEW-3	Changing what players receive (re-upload, or editing the game URL) sends the listing back for re-review.
R-REVIEW-4	A `fail` scan verdict cannot be approved by an admin. **Only a superuser may override**, and the override is recorded.
R-REVIEW-5	Every decision appends an immutable row to `game_reviews`.

Sessions — `game-session.service.ts`

Rule	Statement
R-SES-1	Only approved + enabled games can have sessions created.
R-SES-2	`start_time` must be strictly before `end_time`.
R-SES-3	A player with `player_status = false` cannot join.
R-SES-4	A player cannot join a session they are already in.
R-SES-5	A session cannot exceed `maxPlayers`.
R-SES-6	Only the host (`created_by`) can end a session.
R-SES-7	Once `ended`, no further joins or state changes.
R-SES-8	Ending a session requires at least one result entry.
R-SES-9	On end, `match_results` are written and player stats updated.
R-SES-10	A player may spectate any waiting/active session they are **not** a member of.
R-SES-11	Quick Play joins the busiest available room, or creates one if none exist.

Tournaments — `tournament.service.ts`

Rule	Statement
R-TRN-1	Tournament name is unique.
R-TRN-2	`start_date ≤ end_date`; `regClose ≤ start_date`.
R-TRN-3	Registration only for `upcoming` tournaments, before `regClose`.
R-TRN-4	No double registration.
R-TRN-5	Cannot exceed `maxParticipants`.
R-TRN-6	Only players with `player_status = true` may register.
R-TRN-7	A completed tournament cannot be updated.
R-TRN-8	The linked game must be approved and enabled.

Reports and moderation

Rule	Statement
R-RPT-1	Cannot report yourself.
R-RPT-2	Both users must exist.
R-RPT-3	Duplicate open reports rejected.
R-RPT-4	Reason required.
R-RPT-5	Only mods/admins change status.
R-RPT-6	Resolved/rejected cannot re-open.
R-RPT-7	Type from a fixed list of 7.

Rule	Statement
R-MOD-1	Only role 1/2/3 may act.
R-MOD-2	A ban must include a duration.
R-MOD-3	No double simultaneous ban.
R-MOD-4	Ban writes 3 places (see §9.7).
R-MOD-5	Warn/restrict resolves the report.
R-MOD-6	Cannot act against equal or higher role.
R-MOD-7	Violations without a report are valid.

Notifications and match results

Rule	Statement
R-NTF-1	A notification targets a role (broadcast) **or** a user (direct) — not both, not neither.
R-NTF-2/3/4	`toRole`, `toUser` and `fromUserId` must reference real rows.
R-NTF-5/6	A user may only mark/delete their own notifications.
R-NTF-7	Replies are appended with a timestamp.
R-NTF-8	Read state on a broadcast is **per user** (`readBy[]`) because the broadcast is one shared row.
R-NTF-9	A broadcast may only carry text true for **every** user in the role. Anything phrased "your game / your report" must be one direct row per recipient.
R-MR-1	`(session_id, player_id)` must be unique.
R-MR-2/3	`score ≥ 0`; `player_rank ≥ 1`.
R-MR-4	Match results are query-only — they are written by session end, never edited by hand.

PART 14

Security decisions

Seven real holes that existed in this project, how each was closed, and what is still open. This part is the difference between "I built an app" and "I understand what I built".

14.1 Seven holes and their fixes

① Every visitor received every password

Was	`GET /db-export` returned the full `users` array including plaintext `password`, on an unauthenticated endpoint the front-end calls on **every page load**. All 24 accounts, including the superuser.
Why	Because the login form compared passwords *in the browser*, so it needed them there.
Now	`app.controller.ts:42` — `users.map(({password, ...safe}) => safe)`. Comparison moved to `POST /auth/login`. `db-import` carries the stored password forward whenever a client omits it, so nothing is lost.

② A blank password was a master key

Was	`if (user.password && user.password !== pEl.value)` — an account row with no password skipped the check, so **any** password signed you in as that user.
Now	A missing password is a failure to authenticate, never a pass.

③ One curl command made you superuser

Was	`RolesGuard` trusted `headers['role']` — written by the caller. `curl -H 'role: admin' -X PATCH /users/9 -d '{"role_id":1}'`. Verified twice against this server.
Now	Three layers: the role comes from an HMAC payload the server signed at login; the three privileged roles can *never* be granted by the header; and `db-import` pins `role_id` / `status` to what the server holds unless a staff token is present.

④ Inventing a superuser instead of promoting one

Was	Even after ③, `db-import` could *create* a brand new user row with `role_id: 1`, sidestepping the promote check entirely.
Now	`app.controller.ts:172` — a row with no existing counterpart is clamped to role 4 unless the caller is staff. **The lesson: a rule enforced at one entry point is not enforced.**

⑤ Publishing a game nobody had reviewed

Was	The legacy admin pages set `g.status = 'approved'` in localStorage and pushed the whole DB — publishing a build nothing had looked at.
Now	`app.controller.ts:287` refuses client-side **promotion** to approved. De-escalation is still allowed on purpose: a publisher resubmitting a rejected game sends it back to `pending`, and a game going offline is the safe direction. **Only the promote path is guarded.**

⑥ Locking someone out of their own account in two clicks

Was	The profile page changed a password without asking for the current one. Anyone reaching an open session — shared laptop, borrowed phone — could take the account.
Now	R-AUTH-6: `POST /auth/change-password` requires and verifies `current_password` server-side, and clears `active_session_token` so every other device is evicted. The error message is identical for "no such user" and "wrong password", so the endpoint cannot be used to enumerate user ids.

⑦ Any tab could post a win into the leaderboard

Was	The wrapper accepted any `message` event.
Now	Two checks: `ev.source === frame.contentWindow` and `ev.origin === resolved.origin`. Plus outcome whitelisting and the dropping of game-private economy fields.

14.2 What is still open — and how you would fix it

Weakness	Consequence	The fix you would describe
Passwords stored in plaintext	Anyone who reads `live_database.json` has every credential.	`bcrypt.hash(password, 10)` on register; `bcrypt.compare()` on login. A ~20-line change confined to `auth.service.ts` and `auth.repository.ts`.
Player/publisher routes still accept the `role` header	Someone can call a player-level endpoint without signing in.	Migrate those pages to send the Bearer token, then delete branch 2 of `RolesGuard`.
Hosted bundles run same-origin	An uploaded game can read `localStorage`, including the session.	Serve `games/` from a separate port or subdomain so the iframe is genuinely cross-origin.
Whole-DB sync	Bandwidth grows with the database; concurrent writers rely on the union to not lose data.	Move each page to the specific REST endpoint that already exists for it. The API is complete — the front-end simply does not use most of it.
`origin: '*'` in CORS	Any website can call the API from a user's browser.	Whitelist the actual front-end origins.
No rate limiting	Login can be brute-forced.	`@nestjs/throttler` on `/auth/login`.
`innerHTML` with user text	A username or chat message containing `<script>` could execute (stored XSS).	Use `textContent` for user-supplied strings, or escape before interpolating.

This table is your best viva material Being asked "what is wrong with your project?" is an opportunity, not a trap. A student who says "nothing" sounds naive. A student who says "*passwords are plaintext; I would use bcrypt, and the change is confined to these two files*" sounds like an engineer. Learn three rows from this table by heart.

PART 15

Practice

You said you learn by doing. This is the doing. Work top to bottom — each exercise assumes the previous ones.

15.1 Twenty-five exercises, in difficulty order

Level 1 — Observe (no code changes)

#	Task	What it teaches
1	Start both servers. Open the console and run `window.PC_API`, `DB.users.length`, `Auth.getSession()`.	The three globals everything else is built on.
2	Open DevTools → Application → Local Storage. Find `playcarys_db_store_v5`, `playcarys_session`, `playcarys_api_token`.	Where the browser copy of everything lives.
3	Open the Network tab, click any button that saves, and find the `db-import` request. Look at its size.	That the **whole** database goes over the wire every time.
4	Open `http://localhost:3000/api/docs` and execute `GET /games` from the page.	Swagger, and that the API works independently of the UI.
5	Open `front-end/live_database.json` in VS Code. Click something in the app. Watch the file change on disk.	The full chain: click → memory → HTTP → memory → disk.

Level 2 — Trace

#	Task	What it teaches
6	Put a `debugger;` at the top of `doPanelLogin` and step through a whole login.	The 7-step auth flow, felt rather than read.
7	Add `console.log('[SAVE]', new Error().stack)` inside `DB.save()`. Use the app for two minutes.	How often it fires, and from where. This is genuinely surprising.
8	Open two windows as two accounts. Send a friend request. Watch the request in one and `playcarys:dbupdate` fire in the other.	The real-time round trip.
9	Stop the back-end and use the app. Note exactly which features still work.	Which behaviour is local-only vs server-dependent.
10	Set `DB.users[7].lastSeen = new Date(Date.now() - 5*60*1000).toISOString()` then re-render the friends list.	The presence algorithm and its 2-minute window.

Level 3 — Small changes

#	Task	What it teaches
11	Change `--accent-cyan` in `css/variables.css`. Reload.	How far one custom property reaches.
12	Add a genre in `publisher/manage-genres.html` and confirm it appears in the publish wizard and the games filter.	How `DB.genres` feeds three pages.
13	Add a "Copy session code" button to the room lobby. Follow the 6-step recipe from §5.1.	Your first real feature.
14	Add a new tier above Master in `Economy.POINT_TIERS`.	Where the tier system is single-sourced.
15	Add a new report type to `VALID_REPORT_TYPES` in `report.service.ts` and to the dropdown in `player/profile.html`.	That a rule usually has a front-end half and a back-end half.

Level 4 — Break it deliberately

#	Task	What it teaches
16	Change `sameId()` to `return a === b;`. Use the app as a registered (non-seed) account.	Why string/number id mixing was the project's most common bug class. **Revert afterwards.**
17	Remove `DB.save()` from `doPanelLogin` and try to log in.	The "signed out immediately after signing in" bug, reproduced on purpose.
18	Remove `users` from the `APPEND_ONLY` map. Register an account in window A, then save from a stale window B.	How four real accounts were destroyed.
19	Try to approve a game by setting `game_approval_status='approved'` in the console and calling `DB.save()`. Then read the server console.	The `db-import` promote guard, live. You will see the refusal warning.
20	Delete `back-end/.auth-secret` and restart the API.	Why the key is persisted rather than regenerated. **All staff sessions die.**

Level 5 — Build something real

#	Task	What it teaches
21	Add a `GET /games/:id/stats` endpoint returning sessions, unique players and average score. Full 5-file module pattern.	The whole NestJS pattern, written by you.
22	Hash passwords with `bcrypt`. Change only `auth.service.ts` and `auth.repository.ts`. Migrate the seed rows.	Fixing the project's biggest real weakness.
23	Make `player/games.html` load its data from `GET /api/v1/games?active=true` instead of the local `DB`.	The migration path away from whole-DB sync.
24	Add a third-party game: write a tiny HTML5 game, include the SDK, call `submitResult()`, zip it, publish it, approve it, play it.	The **entire pipeline** end to end. This is the single most valuable exercise in the guide.
25	Add one new scanner rule to `scan.service.ts` — e.g. flag `navigator.geolocation` as medium. Upload a bundle that triggers it.	How the review evidence is produced.

15.2 Viva questions, with answers

"Walk me through what happens when a user logs in."

Recite §9.1. Hit these five beats: local lookup and per-panel role check → `POST /auth/login` where the **server** compares the password → the server signs an HMAC-SHA256 token carrying the role → single-session claim, with a device-handoff request over Socket.IO if the account is already in use → `Auth.login()` writes the session, `DB.save()` pushes the claim, then redirect. Add: *"the password is checked server-side because the original client-side check forced `db-export` to ship every password to every visitor."*

"Why did you not use a real database?"

Be direct: the schema was designed as 12 SQL tables and the code mirrors it exactly — same names, same keys, same foreign-key relationships. It is implemented as TypeScript arrays with a repository layer in front, so **migrating to PostgreSQL means rewriting twelve repository files and nothing else**. No controller, no rule and no DTO would change. That was a deliberate boundary, not an accident.

"How do two players on two laptops stay in sync?"

Three layers. **(1)** Any write calls `DB.save()`, which POSTs the whole database and then emits `global_db_update`. **(2)** The gateway broadcasts `global_db_update_received`; every other client runs `PlayCarysLive.pull()` → `GET /db-export` → `mergeRemoteState()`, then dispatches a **synthetic `storage` event** so every listener that was already written for cross-tab sync starts working across the network unchanged. **(3)** If the Socket.IO CDN never loaded, a 5-second poll does the same job.

"What stops a player from making themselves an admin?"

Nothing on the front-end — and that is correct, because front-end checks are UI convenience, not security. The enforcement is: `RolesGuard` trusts **only** an HMAC-signed token issued at login; the three privileged roles can never be granted by the client-written `role` header; and `db-import` pins `role_id` and `status` to the server's values unless the caller presents a staff token — including for newly created rows, so you cannot invent a superuser either.

"Why is `match_results` append-only?"

Because a match you played is a historical fact, and every client uploads its whole database. A tab holding a stale snapshot was deleting matches another tab had just recorded — server-side, and then in every other client via the broadcast. Reproduced: a run saved successfully, an unrelated save followed, and the run was gone from both the store and the JSON file. The fix is a **union** keyed on `(session_id, player_id)`.

"How is a publisher's game prevented from stealing data?"

Four layers. **(1)** Archive validation — zip-slip, zip bombs, blocked extensions, blocked server-config filenames. **(2)** A static content scan producing evidence an admin reads, with critical rules (reading the session key, cryptominers, scripted SVGs) that fail outright. **(3)** The postMessage bridge validates `ev.source` and `ev.origin`, whitelists outcomes and drops game-private economy fields. **(4)** Self-hosted builds run in a sandboxed iframe. Then add the honest caveat: hosted bundles currently run same-origin, which is a documented limitation — the fix is serving them from a separate origin.

"What was the hardest bug?"

The App-level answer is the **stale-snapshot overwrite**: because every client POSTs the entire database, the last writer won, and a tab that had loaded before a change silently deleted it. It manifested as five unrelated symptoms — vanishing matches, deleted accounts, disappearing friend requests, resurrected deleted games, brackets that reset on refresh. One root cause, one architectural fix: append-only unions plus explicit tombstones.

"Which part would you rewrite?"

The whole-DB sync. Every REST endpoint the front-end needs already exists — the pages simply do not use them. Migrating page by page to specific endpoints would remove the entire merge/union/tombstone layer, cut bandwidth, and eliminate the last-writer-wins class of bug. I would start with `player/games.html` because it is read-only and therefore lowest risk.

15.3 File index — where to look for what

If you are looking for…	Open
Login, registration, guest mode	`front-end/index.html` lines 1380–1745
Session, tokens, device handoff	`js/utils.js` lines 342–688 (`Auth`)
Presence / online status	`js/utils.js` lines 696–861
Points, tiers, XP, royalties	`js/utils.js` lines 863–962 (`Economy`)
Toast, Modal, Validate	`js/utils.js` lines 964–1142
Navbar and role links	`js/utils.js` lines 1145–1279
Drawer, friends, notifications UI	`js/utils.js` lines 1549–2615
Backend URL auto-detect	`data/db.js` lines 34–54
Seed data, all tables	`data/db.js` lines 56–653
Save / merge / append-only / tombstones	`data/db.js` lines 658–1063
Match recording and Carys formulas	`data/db.js` lines 974–1137
Tournament bracket generation	`data/db.js` lines 1139–1260
Notification visibility helpers	`data/db.js` lines 2509–2551
Game launch resolution	`js/game-launch.js` (whole file)
iframe wrapper + SDK bridge	`pages/player/play.html` lines 100–270
Design tokens	`css/variables.css` lines 1–52
API bootstrap and pipeline	`back-end/src/main.ts`
db-export / db-import (the sync brain)	`back-end/src/app.controller.ts`
All interfaces and the in-memory store	`back-end/src/data/store.ts`
Atomic writes and passthrough	`back-end/src/data/persistence.ts`
HMAC token signing and verification	`back-end/src/auth/token.util.ts`
Authorisation guard	`back-end/src/roles/roles.guard.ts`
Bundle upload and zip safety	`back-end/src/game/bundle.service.ts`
Static security scanner and its rules	`back-end/src/game/scan.service.ts`
Socket.IO gateway	`back-end/src/game-session/game.gateway.ts`

A closing note

Almost every design decision in this project came from a **failure that was actually observed**, not from theory. The append-only tables exist because four accounts were destroyed. The signed token exists because a player was escalated to superuser with one curl command. The synthetic `storage` event exists because a guest sat in a lobby forever while the host had already started the match. The scanner exists because a real submission imported a module from a third-party URL at boot.

That is what makes this codebase worth learning from — and it is what you should say when someone asks you to explain it. Not "*I used NestJS and vanilla JavaScript*", but "*here is a problem the system had, here is why the obvious fix does not work, and here is what we do instead.*"

> **Where to start, right now** Do exercises 1–5 today. They take twenty minutes and they turn everything in this document from information into something you have seen with your own eyes.